

The Simultaneous Kalman Smoother: State Space Models Made Easy

Mico Mrkaic
International Monetary Fund
`mmrkaic@imf.org`*

June 2026

Abstract

The Kalman filter and the Rauch–Tung–Striebel smoother are usually derived through a forward–backward recursion of conditional means and covariances. For economists this derivation is needlessly hard to follow. The recursive form has two original motivations, neither of which now applies to us. First, it was developed for online vehicle guidance on 1960s computers with kilobytes of memory, where storing the joint distribution of an entire latent path was impossible. Second, it is in the idiom of control engineering — state machines, predictions, online updates — whereas econometrics is built around joint distributions, regressions, and batch data. This note derives the smoother in the idiom that is native to us. The key observation is that stacking the full latent path and all observations produces one big multivariate normal vector, and the entire Kalman smoother is just the textbook conditional-Gaussian formula applied to that vector. Solving the resulting system in precision form is one sparse block-tridiagonal linear equation $J\hat{X} = h$. The Kalman filter and the RTS smoother are then exposed as the block-LDL[⊤] forward sweep and the selected-inversion backward sweep of the same system — implementation details, not theory. Posterior covariances, marginal likelihood, missing data, diffuse priors, EM, the Hodrick–Prescott filter, and exogenous inputs all become small edits to J and h . Heavy-tailed disturbances (Student- t , Laplace, Huber) become iteratively reweighted versions of the same sparse solve, and the framework extends cleanly to regime-switching dynamics (variational EM on the same J) and to inequality-constrained smoothing (interior-point QP on the same J). The precision-based view is not new; it appears in spatial statistics and Bayesian econometrics as the “Gaussian Markov random field” view of state space. It is, however, considerably easier to teach.

1 An Example to Motivate the Discussion

Most economists meet the Hodrick–Prescott filter as a recipe: to extract a trend τ_t from a series y_t , solve

$$\min_{\tau_1, \dots, \tau_T} \sum_{t=1}^T (y_t - \tau_t)^2 + \lambda \sum_{t=2}^{T-1} [(\tau_{t+1} - \tau_t) - (\tau_t - \tau_{t-1})]^2, \quad (1)$$

with $\lambda = 1600$ for quarterly data, and read off $\hat{\tau}$. The first sum keeps the trend close to the data; the second penalizes changes in the slope of the trend, with λ governing how hard. Most users

*This note and the accompanying `sinksmoother` software package (Octave/MATLAB and Python) were produced under the author’s guidance and direction, with the assistance of AI (Claude, Anthropic). The author is responsible for all content, errors, and omissions.

stop there. But where does this objective come from, what model does it correspond to, and what is λ , really? Answering those questions is the cleanest possible introduction to everything in this note, because the HP filter turns out to be a one-line state-space model, and (1) turns out to be a conditional expectation in a multivariate normal. Once that is clear, a whole catalogue of extensions opens up almost for free.

1.1 The HP filter is a state-space model

Write potential output τ_t as a latent process whose *second difference* is white noise, and observed output y_t as potential plus a transitory gap:

$$(\tau_{t+1} - \tau_t) - (\tau_t - \tau_{t-1}) = e_t, \quad e_t \sim N(0, \sigma_e^2), \quad (2)$$

$$y_t = \tau_t + \varepsilon_t, \quad \varepsilon_t \sim N(0, \sigma_\varepsilon^2). \quad (3)$$

Equation (2) is the economic content of the HP filter, usually left implicit: *potential output is an $I(2)$ process*. Its level is non-stationary, its growth rate $g_t \equiv \tau_t - \tau_{t-1}$ is non-stationary, but the *change* in its growth rate is unforecastable noise. Equation (3) is the trend-cycle decomposition: the part of output we observe is potential plus a transitory deviation ε_t , the “output gap” up to sign.

This is already a useful step, independent of anything that follows. The recipe (1) hides a strong, testable economic assumption — that potential output is $I(2)$ — behind a smoothing objective. Writing the model in the form (2)–(3) makes the assumption explicit, and immediately invites the question of whether it is the right one for a given series. If potential growth is better described as *stationary* ($I(1)$ in the level) rather than as a driftless random walk ($I(2)$ in the level), the HP filter is misspecified, and one would want a different state equation. One cannot even ask that question from the recipe alone.

1.2 Where λ comes from

The two innovation variances are not separately identified by the *shape* of the extracted trend — only their ratio is. To see this, write the joint Gaussian log-density of the latent path and the data implied by (2)–(3). Dropping constants, minimizing $-2 \log p$ over τ gives

$$\min_{\tau} \frac{1}{\sigma_\varepsilon^2} \sum_{t=1}^T (y_t - \tau_t)^2 + \frac{1}{\sigma_e^2} \sum_{t=2}^{T-1} [(\tau_{t+1} - \tau_t) - (\tau_t - \tau_{t-1})]^2. \quad (4)$$

Multiplying through by σ_ε^2 leaves the minimizer unchanged and produces (1) exactly, with

$$\boxed{\lambda = \frac{\sigma_\varepsilon^2}{\sigma_e^2}}. \quad (5)$$

So λ is not a free smoothing knob: it is the ratio of the measurement-noise variance to the trend-innovation variance. The famous value $\lambda = 1600$ for quarterly data is an assertion that the transitory component of output is 1600 times as variable as the shocks to trend growth. Stated that way, it is a falsifiable claim about the data, not a tuning choice — and indeed σ_ε^2 and σ_e^2 can be estimated by maximum likelihood rather than assumed, which is one of the extensions below.

1.3 The same problem in simultaneous form

Now stack. Collect the trend into $\boldsymbol{\tau} = (\tau_1, \dots, \tau_T)'$ and the data into $\mathbf{y} = (y_1, \dots, y_T)'$. The measurement equation (3) stacks to

$$\mathbf{y} = \boldsymbol{\tau} + \boldsymbol{\varepsilon}, \quad \boldsymbol{\varepsilon} \sim N(0, \sigma_\varepsilon^2 I_T). \quad (6)$$

The state equation (2) is a statement about the second difference of $\boldsymbol{\tau}$. Let D_2 be the $(T-2) \times T$ second-difference operator, the matrix whose t -th row places $(1, -2, 1)$ in columns $t, t+1, t+2$:

$$D_2 = \begin{pmatrix} 1 & -2 & 1 & & \\ & 1 & -2 & 1 & \\ & & \ddots & \ddots & \ddots \end{pmatrix}, \quad D_2 \boldsymbol{\tau} = \mathbf{e}, \quad \mathbf{e} \sim N(0, \sigma_e^2 I_{T-2}). \quad (7)$$

Both (6) and (7) are linear in $\boldsymbol{\tau}$ with Gaussian right-hand sides. Reading off the pieces, $\boldsymbol{\tau}$ has prior precision $\sigma_e^{-2} D_2' D_2$ (this is what the $I(2)$ assumption contributes), and the data contributes measurement precision $\sigma_\varepsilon^{-2} I_T$. One qualification is needed: $D_2' D_2$ has rank $T-2$, so the prior on $\boldsymbol{\tau}$ is an *intrinsic* (improper) Gaussian — it is flat in the two directions spanned by the initial level and slope of the trend, the diffuse-limit prior on those components. Strictly, then, $(\boldsymbol{\tau}, \mathbf{y})$ is jointly Gaussian in the intrinsic/improper sense rather than a proper multivariate normal. None of what follows is affected: the *posterior* of $\boldsymbol{\tau}$ given $\mathbf{y}$ is a proper Gaussian, because the measurement term adds the full-rank precision $\sigma_\varepsilon^{-2} I_T$.

The trend we want is the conditional expectation $\mathbb{E}[\boldsymbol{\tau} \mid \mathbf{y}]$, which for a multivariate normal is the familiar Bayesian-GLS / ridge estimator. Equivalently it is the mode, which solves the penalized least-squares problem obtained by stacking (6)–(7):

$$\hat{\boldsymbol{\tau}} = \mathbb{E}[\boldsymbol{\tau} \mid \mathbf{y}] = \arg \min_{\boldsymbol{\tau}} \frac{1}{\sigma_\varepsilon^2} \|\mathbf{y} - \boldsymbol{\tau}\|^2 + \frac{1}{\sigma_e^2} \|D_2 \boldsymbol{\tau}\|^2. \quad (8)$$

Differentiating and setting the gradient to zero gives a single linear system,

$$\underbrace{(I_T + \lambda D_2' D_2)}_J \hat{\boldsymbol{\tau}} = \underbrace{\mathbf{y}}_h, \quad \lambda = \frac{\sigma_\varepsilon^2}{\sigma_e^2}, \quad (9)$$

where we divided through by σ_ε^{-2} so that only the ratio λ appears. This is the closed-form HP solution $\hat{\boldsymbol{\tau}} = (I + \lambda D_2' D_2)^{-1} \mathbf{y}$ that appears in every reference — but now we have *derived* it, as the conditional mean of a multivariate normal, rather than posited it as an objective.

Three things are worth pausing on, because they are the whole note in miniature.

First, the matrix $J = I + \lambda D_2' D_2$ is *banded*: D_2 has bandwidth two, so $D_2' D_2$ is pentadiagonal and J is pentadiagonal. Solving (9) by a banded Cholesky factorization costs $O(T)$ operations and $O(T)$ storage, not the $O(T^3)$ a dense $(I + \lambda D_2' D_2)^{-1}$ would suggest. The sparsity is not an accident of the HP filter; it is the stacked image of the fact that τ_t interacts only with its near neighbours in time, and it is exactly the structure that makes the general simultaneous smoother of this note cheap.

Second, the recipe and the model are now visibly the same object. The HP “smoothing parameter” is a variance ratio; the “smoothing penalty” is the $I(2)$ prior; the “filter” is a conditional expectation in a Gaussian. No forward pass, no backward pass, no Kalman gain — just $J\hat{\boldsymbol{\tau}} = h$, one sparse solve.

Third, and most important for what follows, this derivation generalizes immediately. We made no use of the specific structure of the HP filter beyond “the latent path is constrained by a linear

Gaussian state equation, and the data is a linear Gaussian readout of the state.” Any linear Gaussian state-space model has that form, so any such model reduces to a sparse linear system $J\hat{X} = h$ in exactly the same way. The rest of this note works out the general case.

1.4 What this buys us

Because the HP filter is now an instance of one Gaussian conditioning problem rather than a standalone recipe, extensions that would each require their own derivation in the recursive idiom become small, composable edits to J and h . A few that we develop later, each already meaningful for the trend-cycle decomposition above:

- **Exogenous drivers.** Suppose a credit gap or a terms-of-trade index z_t shifts trend growth. Add βz_t to the state equation (2) or γz_t to the measurement (3). In simultaneous form this leaves J *unchanged* and only shifts h by a deterministic offset (Section 3). The smoothed trend then decomposes additively into a “pure HP” part and a credit-driven part.
- **Sign and shape constraints.** Suppose we want to impose that potential *growth* never turns negative, $g_t = \tau_t - \tau_{t-1} \geq 0$, or that a NAIRU-like latent process stays non-negative. In the recursive form there is no clean way to do this; the Kalman recursion produces an unconstrained Gaussian. In simultaneous form it is a single inequality-constrained quadratic program with the *same* sparse J as its Hessian, solved by an interior-point method (Section 7.3).
- **Estimating λ instead of asserting it.** Since $\lambda = \sigma_\varepsilon^2 / \sigma_e^2$ is a likelihood parameter, we can estimate it — and hence let the data choose the degree of smoothing — by maximizing the marginal likelihood, which the simultaneous form computes from the same factorization of J that delivers the trend (Sections 4.6 and 6).
- **Robustness to outliers and breaks.** A one-off oil shock or a pandemic quarter is a fat-tailed measurement error; a structural break is a fat-tailed trend innovation. Replacing the Gaussian errors with Student- t errors turns (8) into an iteratively-reweighted version of the same sparse solve (Section 7.1), down-weighting the offending observations automatically.
- **Honest uncertainty bands.** The conditional *variance* $\text{Var}[\boldsymbol{\tau} \mid \mathbf{y}]$ gives pointwise error bands on the trend. In the normalization of (9), where $J = I + \lambda D_2' D_2$ after dividing through by σ_ε^{-2} , the posterior covariance is $\text{Var}[\boldsymbol{\tau} \mid \mathbf{y}] = \sigma_\varepsilon^2 J^{-1}$ — the measurement-noise scale multiplies the normalized inverse (only if one imposes $\sigma_\varepsilon^2 = 1$ is it J^{-1} itself). The scale does not affect the smoothed trend, but it does set the width of the bands. These widen automatically where data is missing and narrow where it is dense, and they are computed without forming the dense inverse, by a sparse selected-inversion recursion (Section 4.5).

Every one of these is a small modification of the same object $J\hat{X} = h$. None requires a new filter, a new backward pass, or a re-derived boundary condition. This composability — many extensions, one sparse linear system — is the practical case for the simultaneous formulation, on top of the formulation being, in our experience, much easier to understand. The remainder of the note makes the general construction precise and then returns, in the worked applications of Appendix C, to exactly the kind of trend-cycle problem this section started with.

2 A More Intuitive Way to Derive the Kalman Filter

The Kalman filter and the Rauch–Tung–Striebel (RTS) smoother¹ as they appear in econometrics textbooks descend, almost unaltered, from Kalman (1960) and from Rauch et al. (1965). Those papers solved a specific engineering problem: online estimation of the state of a vehicle from a stream of noisy sensor readings, on hardware with kilobytes of memory. Two constraints shaped the derivation.

First, the problem was *online* — meaning it had to be solved in real time. A new measurement arrived at each instant and the estimate had to be updated immediately, without revisiting the past. A forward recursion of conditional moments is the right object for that setting; it is what you would invent if you had to.

Second, the hardware could not store the past. RAM was scarce and expensive — the Apollo Guidance Computer, designed around the same time, had two kilobytes of erasable memory. Holding the joint distribution of an entire latent path was inconceivable. The forward recursion was not just elegant; it was the only thing that fit. The RTS smoother of 1965 added a single backward pass for the same reason: a second forward sweep would have required storing more than the memory budget allowed.

Neither constraint binds in modern macroeconometrics. We are not doing online state estimation on a missile. We have a batch of data, on disk, and we want the smoothed path, the parameters, posterior bands, and the marginal likelihood jointly, in one shot. A laptop holds gigabytes; the joint precision matrix² for a model with $n = 20$ states and $T = 200$ has on the order of 1.6×10^5 nonzeros,³ which is negligible.

Econometric tradition and methodology differ from those used in engineering. Control engineers think in terms of state machines, predictions, and online updates. Economists think in terms of joint distributions, regressions, OLS and GLS, and matrix algebra. The recursive derivation is in the wrong idiom for us. It asks the student to translate econometric questions — what is the posterior mean of x_t given all observations? what is the marginal log-likelihood? what changes when an observation is missing? — into a foreign language about filters and predictions, and then translate the output back. The simultaneous derivation does not require this. It lives in the native econometric idiom: stack the latent path and the observations into one big multivariate normal vector, and apply the standard conditional Gaussian formula. That formula is, structurally, the GLS estimator; every economist already knows it. Conditional moments, regression interpretations, GLS analogies, penalized least squares, and likelihood are all visible at a glance.

This is the motivation. The recursive form is not wrong, and remains the right tool for genuine online filtering. But for the kind of batch work that applied macroeconomists actually do, it is needlessly indirect, and teaching it as the primary derivation propagates that indirectness to each new cohort of students.

The benefits of the simultaneous approach extend well beyond ease of understanding. By placing everything inside one Gaussian conditioning problem, it produces a unified framework in which extensions of the basic Gaussian Kalman filter — heavy-tailed disturbances, regime-switching

¹Often called simply the “Kalman smoother” in economics texts. We retain the RTS terminology to keep the filter and smoother distinct.

²Throughout the note the *precision matrix* of a Gaussian random vector means the inverse of its variance–covariance matrix. Econometrics typically works with covariances directly; the precision form is what makes the sparsity of state-space models visible and the solves fast, since the inverse of a dense covariance can be a sparse matrix and vice versa.

³The precision matrix is block tridiagonal with $T + 1$ diagonal blocks and T off-diagonal blocks, each of size $n \times n$. Counting upper-triangular storage of the symmetric matrix, the structural nonzero count is $(T + 1)n^2 + Tn^2 = (2T + 1)n^2$; for $T = 200$ and $n = 20$ this gives $401 \times 400 = 160\,400$.

dynamics, inequality constraints, missing data, diffuse priors, time-varying parameters, exogenous inputs, and EM estimation of unknown parameters — become small edits to the same sparse linear system, rather than separate derivations to be maintained independently. This is what makes the framework not just easier to teach, but easier to extend in practice.

2.1 Where the simultaneous formulation is already standard

Economists’ near-universal use of the recursive form is, by a wide margin, the historical outlier. In almost every neighbouring discipline that handles latent Gaussian fields at scale, the precision-form / simultaneous representation has been the standard production tool for years, leveraging the weapons-grade sparse linear algebra infrastructure — CHOLMOD, MUMPS, PaStiX, SuiteSparse, Eigen — that the numerical-analysis community has built over four decades. A non-exhaustive map of where this idiom already lives:

- **Spatial statistics and the INLA ecosystem.** The *integrated nested Laplace approximation* (Rue, Martino, and Chopin, 2009) treats spatial, temporal, and spatio-temporal models as Gaussian Markov random fields with sparse precision matrices and solves them with sparse Cholesky. R-INLA has been the dominant Bayesian inference tool for latent Gaussian models in epidemiology, ecology, disease mapping, and spatial econometrics for fifteen years. The underlying engine is exactly the $JX = h$ solve developed here. Rue and Held (2005) is the canonical reference and motivates the representation directly.
- **Modern Bayesian samplers.** When Stan (Carpenter et al., 2017) runs Hamiltonian Monte Carlo on a state-space model with a Gaussian prior over the latent path, the log-density and gradient computations are simultaneous-form operations on the sparse precision matrix; the autodiff system never sees a forward-backward recursion. Every Stan model with a latent Gaussian field is implicitly doing simultaneous-form inference at production scale.
- **Large Bayesian VARs and big-data macro.** Banbura and Modugno (2014) use an essentially simultaneous EM representation for mixed-frequency factor models with arbitrary missing-data patterns, precisely because the recursive form does not scale gracefully to large panels. More recent large-BVAR work (Giannone, Lenza, and Primiceri, 2015) moves in the same direction: once the number of variables exceeds the few-dozen range, the precision-form representation is no longer optional. This is the macro setting closest to the present note, and the direction of travel in the literature is clear.
- **Geophysical data assimilation.** Operational weather prediction at ECMWF and NCEP, ocean reanalysis, and seismic inversion all rely on variational data assimilation (*4D-Var*), which is structurally the same object as the simultaneous smoother: a quadratic penalty on a state trajectory minimized via sparse linear solves. The systems handle state dimensions in the millions on dedicated HPC hardware; none of them runs a recursive Kalman filter as the production solver.
- **Astrostatistics.** Large-scale Gaussian process regression with structured covariances — satellite-data calibration, gravitational-wave detection pipelines at LIGO, exoplanet-search likelihood evaluations — routinely uses precision-form representations with sparse Cholesky. The numerical libraries underneath (CHOLMOD, PaStiX, MUMPS, SuperLU) are the same ones that any production state-space code would use.
- **Image deblurring and structured inverse problems.** The classical Tikhonov regularization of an inverse problem, $\min_X \|y - AX\|^2 + \lambda \|LX\|^2$, is structurally identical to the

simultaneous smoother with $A = \mathcal{H}$ and L encoding the state dynamics. Production image-processing pipelines (medical imaging, satellite imagery) have solved problems in this form for decades.

- **Computational statistics, broadly.** [Chan and Jeliazkov \(2009\)](#) and [McCausland et al. \(2011\)](#) are early macro-statistics references for the precision-form representation of state-space inference, including efficient simulation smoothing for posterior draws. Both papers benchmark the simultaneous form against the recursive form and find the former competitive or faster on realistic problem sizes, contradicting the folk impression that recursion is the high-performance choice.

The common thread is that none of these communities runs a hand-coded Riccati recursion in user-level code. They formulate the latent-Gaussian inference problem as a sparse linear system and hand it to a production-grade sparse linear algebra library. The recursive Kalman filter, viewed from this perspective, is one particular factorization scheme — the block-LDL^T forward sweep developed in Section 4.4 — that an optimizing sparse solver would discover automatically on a block-tridiagonal matrix, without needing to be programmed by hand.

What this means for econometric practice is concrete: when one calls `J \ h` in Octave or `spsolve(J, h)` in SciPy, or invokes any modern sparse-Cholesky routine, one is standing on roughly two hundred person-years of work by serious numerical analysts — much of it specifically tuned for the kind of structured matrices that arise in state-space problems. A hand-coded recursive Kalman loop in `.m` or `.py` gets none of that benefit.

A calibrated statement of the numerical trade-off is in order, because the recursive-form libraries (`statsmodels.tsa.statespace`, KFAS, EViews, Stata’s `sspace`) have had decades of careful engineering invested in making the recursive form robust — square-root forms, exact diffuse initializations, Joseph forms, UDU and Bierman–Thornton variants ([Bierman, 1977](#)) — precisely because naive Riccati updates can lose symmetry or positive definiteness through subtraction of covariance matrices. Sparse Cholesky on J avoids that particular failure mode by construction: J is assembled, not propagated, and the factorization of an SPD matrix never subtracts covariances. But the precision form is not a free lunch. It is a *normal-equations* representation — J contains Q^{-1} , R^{-1} , and products such as $F'Q^{-1}F$ — so its conditioning degrades quadratically where a square-root (factored) representation degrades linearly, and it requires Q and R to be invertible, which models with deterministic state components achieve only through explicit regularization (Section 3.3.1 returns to this). Square-root and UDU filters were invented for genuinely ill-conditioned online problems and retain documented advantages there. The claim of this note is therefore deliberately scoped: *for well-posed batch Gaussian smoothing with an SPD precision matrix* — which covers the textbook econometric use cases — sparse Cholesky on J is a robust, streamlined, and far more teachable coordinate frame, with the production sparse-linear-algebra stack behind it, not an unconditional numerical replacement for every Kalman variant in every regime.

This is the strongest practical case for the simultaneous formulation. The underlying mathematics is identical; the recursive form remains a valid factorization scheme with its own documented strengths; but for the batch problems this note is about, the software stack the rest of computational statistics has converged on assumes the simultaneous representation, and economists who move to that idiom inherit a numerical infrastructure that is far larger, more battle-tested, and faster on real hardware than what any individual research code can hope to match.

3 Model

Consider the linear Gaussian state-space system

$$x_t = Fx_{t-1} + Bz_t + w_t, \quad w_t \sim N(0, Q), \quad t = 1, \dots, T, \quad (10)$$

$$y_t = Hx_t + Dz_t + v_t, \quad v_t \sim N(0, R), \quad t = 1, \dots, T, \quad (11)$$

with initial condition

$$x_0 \sim N(a_0, P_0). \quad (12)$$

Here $x_t \in \mathbb{R}^n$ is latent and $y_t \in \mathbb{R}^m$ is observed. The vector $z_t \in \mathbb{R}^k$ is an observed exogenous input — a deterministic forcing term that enters the state via the loading matrix B (size $n \times k$) and the measurement via D (size $m \times k$). Setting $B = 0$ and $D = 0$ recovers the textbook no-exogenous case.

The two equations describe the same two-layer structure that organizes all linear state-space modelling. Equation (10) is the *state* (or *transition*) equation: the unobserved latent vector x_t evolves autoregressively according to F , driven by exogenous forcing Bz_t and Gaussian innovations w_t . Equation (11) is the *measurement* (or *observation*) equation: the latent state is observed only through the linear combination Hx_t , contaminated by Gaussian noise v_t . Many familiar macroeconometric objects fit this template:

- A univariate AR(1) signal observed with noise has $n = m = 1$, $F = \phi$, $H = 1$;
- The Hodrick–Prescott filter has $n = 2$, $m = 1$, with F encoding a trend and a slope component;
- A small structural VAR has $n = m$ equal to the number of observed variables, with $H = I$ and R small;
- A dynamic factor model has $n \ll m$, with H a tall matrix of factor loadings.

Section 3.3.1 works through four cases in full detail — an AR(1) signal in noise, the HP filter, an MA(1) via state augmentation, and an HP filter driven by an observed financial cycle — and the worked applications of Appendix C include a structural multi-equation system and a dynamic factor model.

Including exogenous inputs from the outset reflects how state-space models are actually used in macroeconometrics: an output gap is estimated jointly with the dynamics of a credit cycle, an unobserved-components model is augmented with policy variables, a factor model carries observed drivers alongside the latent factors. The technical apparatus is identical to the no-exogenous case; the key observation, made precise in Section 4, is that z_t leaves the precision matrix J entirely unchanged and only shifts the information vector h deterministically. This is one of the cleanest payoffs of the simultaneous formulation: the geometry of the smoothing problem is invariant to deterministic forcing.

The goal of smoothing is to estimate the full path

$$X = (x'_0, x'_1, \dots, x'_T)' \in \mathbb{R}^{(T+1)n} \quad (13)$$

using all observations $Y = (y'_1, \dots, y'_T)'$ and all exogenous inputs $Z = (z'_1, \dots, z'_T)'$.

3.1 The state dynamics as one block-bidiagonal system

Define the stacked innovation vector

$$W = \begin{pmatrix} x_0 - a_0 \\ w_1 \\ \vdots \\ w_T \end{pmatrix}, \quad \Omega = \text{Var}(W) = \text{diag}(P_0, Q, \dots, Q). \quad (14)$$

Now define

$$G = \begin{pmatrix} I & 0 & 0 & \dots & 0 \\ -F & I & 0 & \dots & 0 \\ 0 & -F & I & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & 0 \\ 0 & 0 & \dots & -F & I \end{pmatrix}, \quad c_W = \begin{pmatrix} a_0 \\ Bz_1 \\ Bz_2 \\ \vdots \\ Bz_T \end{pmatrix}. \quad (15)$$

The vector c_W collects two things: the prior mean a_0 in the top block, and the deterministic exogenous contributions Bz_t to each state transition. With $B = 0$ the exogenous blocks vanish and we recover the no-exogenous case. Then the full state evolution is simply

$$GX - c_W = W, \quad W \sim N(0, \Omega). \quad (16)$$

This is the first key simplification. The dynamics are not a mysterious recursion. They are one sparse linear equation.

3.1.1 Nilpotent-shift representation

Let $S_\downarrow$ be the lower shift matrix with ones on the first subdiagonal. Then

$$G = I_{T+1} \otimes I_n - S_\downarrow \otimes F. \quad (17)$$

The intuition behind this Kronecker decomposition is that G is the identity minus a one-step-delay shift weighted by F : the diagonal of G holds the identity on each x_t , and the subdiagonal applies $-F$ to map x_{t-1} to its contribution in the t -th equation. Powers of $S_\downarrow$ shift down by more positions, and $S_\downarrow^{T+1} = 0$ because one cannot shift further than the horizon $T + 1$. The Neumann series for G^{-1} therefore terminates at order T :

$$G^{-1} = \sum_{k=0}^T S_\downarrow^k \otimes F^k. \quad (18)$$

The (t, s) block of G^{-1} is therefore

$$(G^{-1})_{ts} = \begin{cases} F^{t-s}, & t \geq s, \\ 0, & t < s. \end{cases} \quad (19)$$

This matrix is the explicit propagation operator: shocks are pushed forward by powers of F , and the lower-triangular block structure encodes the causal arrow of time.

3.2 The measurement equation

Stack observations and exogenous inputs as

$$Y = \begin{pmatrix} y_1 \\ \vdots \\ y_T \end{pmatrix}, \quad Z = \begin{pmatrix} z_1 \\ \vdots \\ z_T \end{pmatrix}. \quad (20)$$

Define

$$\mathcal{H} = \begin{pmatrix} 0 & H & 0 & \cdots & 0 \\ 0 & 0 & H & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & H \end{pmatrix}, \quad \mathcal{D} = \text{diag}(D, D, \dots, D), \quad \Sigma = I_T \otimes R. \quad (21)$$

Then

$$Y = \mathcal{H}X + \mathcal{D}Z + V, \quad V \sim N(0, \Sigma). \quad (22)$$

The first block-column of $\mathcal{H}$ is identically zero: x_0 never enters any observation equation, since measurements run from $t = 1$. The initial state is constrained only by the prior $x_0 \sim N(a_0, P_0)$, which contributes the P_0^{-1} term to the top-left block J_{00} of the precision matrix. No $H'R^{-1}H$ term ever lands there.

3.3 The stacked model as one big multivariate normal

We now have the entire model in two lines:

$$GX - c_W = W, \quad (23)$$

$$Y - \mathcal{H}X - \mathcal{D}Z = V, \quad (24)$$

with $W \sim N(0, \Omega)$, $V \sim N(0, \Sigma)$, and W independent of V . Both equations are linear, and the right-hand sides are jointly Gaussian. The exogenous input Z is observed and deterministic, so it enters only through means and not variances. Therefore (X, Y) is itself a multivariate normal vector of dimension $(T+1)n + Tm$, and its mean and covariance follow by inspection.

The marginal mean of X is the deterministic propagation of the initial mean and the exogenous forcing,

$$a = \mathbb{E}X = G^{-1}c_W, \quad \text{i.e.} \quad a_t = F^t a_0 + \sum_{s=1}^t F^{t-s} B z_s, \quad t = 0, \dots, T. \quad (25)$$

The first term is the textbook propagation; the second is the controllability-style integral that maps the exogenous history into the state-mean path. The marginal covariance of X is the same as in the no-exogenous case,

$$\Sigma_X = \text{Var } X = G^{-1} \Omega G^{-\top}, \quad (26)$$

because Z is observed and deterministic, so it contributes only to the mean. The joint mean and covariance of (X, Y) are then

$$\mathbb{E} \begin{pmatrix} X \\ Y \end{pmatrix} = \begin{pmatrix} a \\ \mathcal{H}a + \mathcal{D}Z \end{pmatrix}, \quad \text{Var} \begin{pmatrix} X \\ Y \end{pmatrix} = \begin{pmatrix} \Sigma_X & \Sigma_X \mathcal{H}^\top \\ \mathcal{H} \Sigma_X & \mathcal{H} \Sigma_X \mathcal{H}^\top + \Sigma \end{pmatrix}. \quad (27)$$

The covariance is identical to the no-exogenous case; only the mean acquires the deterministic offset from Z . That is the whole model.

It is worth pausing to emphasize: equation (27) is the fundamental object of this entire note. Everything that follows — the Kalman filter, the RTS smoother, the marginal log-likelihood, EM, the precision-form sparsity, missing data, diffuse priors, robust extensions, regime-switching, constrained smoothing — is a corollary of (27) obtained by applying the standard conditional-Gaussian formula or by reorganizing the computation to exploit sparsity. In particular, the usual econometric intuition takes over immediately: X is unobserved, Y is observed, and the operation that recovers the unobserved variable is regression of X on Y . Carrying out that regression on the joint normal in (27) is precisely what produces the Kalman smoother in the next subsection.

3.3.1 Four examples to ground the abstraction

Before applying the conditioning formula, four classical models make the abstract $G, \Omega, \mathcal{H}, \Sigma$ concrete. Each is a textbook state-space system written first in its conventional recursive form and then in our stacked matrix form. The mapping is mechanical: pick F, H, Q, R from the recursive equations, and the four big matrices follow.

Example 1: AR(1) signal plus noise. The simplest non-trivial state-space model is a scalar AR(1) latent process observed with noise,

$$x_t = \phi x_{t-1} + w_t, \quad w_t \sim N(0, \sigma_w^2), \quad (28)$$

$$y_t = x_t + v_t, \quad v_t \sim N(0, \sigma_v^2). \quad (29)$$

All four state-space matrices are scalars: $F = \phi$, $H = 1$, $Q = \sigma_w^2$, $R = \sigma_v^2$. The dynamics matrix is bidiagonal and the measurement matrix is a shift of the identity:

$$G = \begin{pmatrix} 1 & 0 & 0 & \cdots & 0 \\ -\phi & 1 & 0 & \cdots & 0 \\ 0 & -\phi & 1 & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & 0 \\ 0 & 0 & \cdots & -\phi & 1 \end{pmatrix}, \quad \mathcal{H} = \begin{pmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \end{pmatrix}, \quad (30)$$

with $\Omega = \text{diag}(P_0, \sigma_w^2, \dots, \sigma_w^2)$ and $\Sigma = \sigma_v^2 I_T$.

It is worth working through how the precision matrix arises from the covariance in this scalar case — the same logic applies in every later example, just with $n \times n$ blocks instead of scalars. From $X = G^{-1}(W + \text{const})$ the marginal covariance of X is $\Sigma_X = G^{-1}\Omega G^{-\top}$, which is dense (it is the classical lag-correlation matrix of an AR(1) plus a top-corner term from the prior). Its inverse, however, is sparse:

$$\Sigma_X^{-1} = G^\top \Omega^{-1} G, \quad (31)$$

and since G is bidiagonal and Ω^{-1} is diagonal, the product $G^\top \Omega^{-1} G$ is tridiagonal. Adding $\mathcal{H}^\top \Sigma^{-1} \mathcal{H}$, which is itself diagonal because measurements do not couple periods, gives

$$J = G^\top \Omega^{-1} G + \mathcal{H}^\top \Sigma^{-1} \mathcal{H}. \quad (32)$$

Carrying out the multiplication block by block yields the explicit form

$$J = \begin{pmatrix} P_0^{-1} + \frac{\phi^2}{\sigma_w^2} & -\frac{\phi}{\sigma_w^2} & & & \\ -\frac{\phi}{\sigma_w^2} & \frac{1+\phi^2}{\sigma_w^2} + \frac{1}{\sigma_v^2} & -\frac{\phi}{\sigma_w^2} & & \\ & \ddots & \ddots & \ddots & \\ & & -\frac{\phi}{\sigma_w^2} & \frac{1+\phi^2}{\sigma_w^2} + \frac{1}{\sigma_v^2} & -\frac{\phi}{\sigma_w^2} \\ & & & -\frac{\phi}{\sigma_w^2} & \frac{1}{\sigma_w^2} + \frac{1}{\sigma_v^2} \end{pmatrix}, \quad (33)$$

with right-hand side $h_0 = P_0^{-1}a_0$ and $h_t = y_t/\sigma_v^2$ for $t \geq 1$. The smoother is a single tridiagonal solve, which is what Thomas's algorithm or the Kalman filter does in this case. Each diagonal entry has a clear interpretation: the prior precision plus the precision of the incoming and outgoing innovations plus the measurement precision. Smoothing is exactly penalized least squares with state innovations penalized at scale $1/\sigma_w^2$ and measurement residuals at scale $1/\sigma_v^2$. The familiar Kalman smoother formulas drop out of this tridiagonal solve.

The general lesson: a dense covariance does not mean a dense calculation, because the inverse of a dense covariance can be a sparse precision and vice versa. Equation (31) is the algebraic fact that makes the simultaneous formulation computationally attractive.

Example 2: the Hodrick–Prescott filter. The HP filter has a two-dimensional latent state: the trend level τ_t and its slope $\Delta\tau_t$. The recursive form is

$$\tau_t = \tau_{t-1} + \Delta\tau_{t-1}, \quad (34)$$

$$\Delta\tau_t = \Delta\tau_{t-1} + e_t, \quad e_t \sim N(0, 1/\lambda), \quad (35)$$

$$y_t = \tau_t + v_t, \quad v_t \sim N(0, 1). \quad (36)$$

Before turning to the state-space matrices, it is worth substituting through the recursion to see what economic content this puts on the trend. From the first equation, $\tau_t - \tau_{t-1} = \Delta\tau_{t-1}$; applying the same identity at $t-1$ gives $\tau_{t-1} - \tau_{t-2} = \Delta\tau_{t-2}$. Subtracting,

$$\Delta^2\tau_t = (\tau_t - \tau_{t-1}) - (\tau_{t-1} - \tau_{t-2}) = \Delta\tau_{t-1} - \Delta\tau_{t-2} = e_{t-1} \sim N(0, 1/\lambda). \quad (37)$$

In other words, the HP filter is the maximum-likelihood smoother for a model in which the *second difference* of the trend is iid Gaussian. When the observed series is interpreted as log output, this means the HP model assumes that potential log GDP is $I(2)$: the trend itself is not stationary, its first difference (trend growth) is not stationary, but the second difference (the change in trend growth) is white noise. This is a strong and testable economic assumption, and it is the right vantage point from which to ask whether the HP filter is the right model for a given series — if the data suggest that potential GDP growth itself is stationary ($I(1)$ rather than $I(2)$ in level), the HP model is misspecified.

With state $x_t = (\tau_t, \Delta\tau_t)^\top$ (dimension $n = 2$), the state-space matrices are

$$F = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \quad H = \begin{pmatrix} 1 & 0 \end{pmatrix}, \quad Q = \begin{pmatrix} \varepsilon & 0 \\ 0 & 1/\lambda \end{pmatrix}, \quad R = 1, \quad (38)$$

where $\varepsilon > 0$ is a tiny regularizer on the level dimension of Q . This deserves a comment, because it has caused confusion in practice. The HP model genuinely has *zero* innovation on the level component — τ_t is determined exactly by τ_{t-1} and the slope $\Delta\tau_{t-1}$, with no own shock — so the natural Q is $\text{diag}(0, 1/\lambda)$, singular. Whether singular Q is a problem depends on which representation of the smoother one uses.

In the conditioning form, equations (48)–(49), the only matrices that need inverting are P_0 and $\mathcal{H}\Sigma_X\mathcal{H}^\top + \Sigma$. Neither requires Q^{-1} ; the joint distribution (27) is perfectly well-defined when Q is singular, since $\Sigma_X = G^{-1}\Omega G^{-\top}$ exists whether or not Ω is invertible. In this representation, ε is not needed.

In the *precision* form (Section 4 below), we work with $J = G^\top\Omega^{-1}G + \mathcal{H}^\top\Sigma^{-1}\mathcal{H}$, which requires Ω^{-1} and therefore Q^{-1} . Here the $\varepsilon > 0$ is needed — not as a modelling assumption, but as a numerical device to make Q invertible. As $\varepsilon \rightarrow 0$ the precision-form smoother approaches the

singular- Q smoother of the conditioning form, and in practice any ε many orders of magnitude below $1/\lambda$ is invisible in the smoothed path.

Honesty requires spelling out what the regularizer does and does not preserve. Inserting ε changes the *model*: the precision-form likelihood of Section 4.6, the EM bound of Section 6, and the conditioning of J are those of the regularized model, not of the exact degenerate-Gaussian model with singular Q . For the smoothed path the difference vanishes as $\varepsilon \rightarrow 0$; for likelihood values it shows up as a $\log \varepsilon$ term in $\log \det \Omega$ that is *constant* in the remaining parameters — so likelihood *ratios*, EM monotonicity, and ML estimates of the non-degenerate parameters are unaffected so long as ε is held fixed, but the likelihood *level* is that of the regularized model. Exact alternatives exist where the distinction matters: eliminate the deterministic state components analytically before assembling J (for the HP filter this is precisely the reduction to the trend-only system $I + \lambda D_2^\top D_2$ of Section 1), or write the degenerate-Gaussian likelihood with pseudo-determinants on the support of Q . The formal statements of this note — the LDL[⊤]/Kalman equivalence, the marginal-likelihood formula, the EM monotonicity — are claims about models with symmetric positive-definite P_0 , Q , and R ; degenerate components are handled either exactly, by elimination, or approximately, by an explicit regularizer whose role is disclosed. The recursive HP smoother makes the same choice implicitly, hidden inside the Riccati recursion; the simultaneous form merely makes the regularization visible because Q^{-1} appears in J literally.

The structure of G , $\mathcal{H}$, Ω , Σ is the same as in Example 1 except that every entry is now a 2×2 block instead of a scalar. The precision matrix J is block tridiagonal with 2×2 blocks. Concentrating out $\Delta\tau_t$ and writing the objective in terms of the trend alone, the Gaussian smoothing problem becomes

$$\min_{\tau} \sum_{t=1}^T (y_t - \tau_t)^2 + \lambda \sum_{t=3}^T (\Delta^2 \tau_t)^2, \quad (39)$$

which is the original Hodrick–Prescott penalized least squares with smoothing parameter λ . (With the convention $\Delta^2 \tau_t = \tau_t - 2\tau_{t-1} + \tau_{t-2}$ adopted above, the penalty runs over $t = 3, \dots, T$, so that every term involves only $\tau_1, \dots, \tau_T$; this is the same $T-2$ curvature terms as the equivalent indexing $\sum_{t=2}^{T-1} (\tau_{t+1} - 2\tau_t + \tau_{t-1})^2$ used in Section 1, and matches the second-difference operator $D_2^\top D_2$.) The classical closed-form solution is $\hat{\tau} = (I + \lambda D_2^\top D_2)^{-1} y$, where D_2 is the second-difference operator.

This is the general lesson: multi-dimensional state is mechanically no harder than the scalar case. The blocks of J are $n \times n$ instead of 1×1 ; the sparsity pattern and the solve are identical. And the HP filter, the Beveridge–Nelson decomposition, unobserved-components models, and the textbook state-space problem are all the same kind of object — a quadratic penalty on a latent path solved by $J\hat{X} = h$. The simultaneous formulation makes that visible.

Example 3: MA(1) via state augmentation. The moving-average model

$$y_t = \epsilon_t + \theta \epsilon_{t-1}, \quad \epsilon_t \sim N(0, \sigma^2), \quad (40)$$

does not fit our framework directly, because y_t depends on both ϵ_t and ϵ_{t-1} while our $\mathcal{H}$ is block diagonal in the state. The textbook fix is to augment the state to carry the lagged innovation:

$$x_t = \begin{pmatrix} \epsilon_t \\ \epsilon_{t-1} \end{pmatrix}, \quad F = \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}, \quad H = \begin{pmatrix} 1 & \theta \end{pmatrix}. \quad (41)$$

The dynamics inject the new innovation into the first component and shift the previous one into the second: $x_t = Fx_{t-1} + (\epsilon_t, 0)^\top$, with $Q = \text{diag}(\sigma^2, \varepsilon)$ for a small regularizer ε . Measurement is essentially noiseless, $R = \varepsilon'$, with a small regularizer for invertibility.

The MA(1) example illustrates a general principle that applies to any ARMA(p, q) process: by augmenting the state to dimension $n = \max(p, q + 1)$, the process fits the framework with F in companion form and Q injecting innovations into the first component. The sparsity of J depends on n , not on the order of the moving average. Long-MA processes get compact sparse formulations. The recursive filter applied to this augmented system is exactly what econometrics textbooks call “the Kalman filter for ARMA models”.

A subtle point worth flagging now and resolving later: the augmented state in (41) is one of many equivalent realizations of the same observed MA(1) process. An MA(1) is identified by the scalar θ alone, but our state representation is identified only up to invertible transformations $x_t \mapsto Mx_t$ with corresponding adjustments to F , H , and Q . This rotational ambiguity is generic; we return to it systematically in Section 3.5. For now it suffices to note that any sensible estimation procedure — the EM algorithm of Section 6 in particular — can only identify the observable invariants of (F, H, Q, R) , not their entries.

Example 4: HP filter with a financial cycle driver. The previous examples had no exogenous inputs. Now consider an applied case where the latent trend in output is pushed by an observed financial cycle, say a credit gap z_t . The recursive form is

$$\tau_t = \tau_{t-1} + \Delta\tau_{t-1}, \quad (42)$$

$$\Delta\tau_t = \Delta\tau_{t-1} + \beta z_t + e_t, \quad e_t \sim N(0, \sigma_e^2), \quad (43)$$

$$y_t = \tau_t + \gamma z_t + v_t, \quad v_t \sim N(0, \sigma_v^2), \quad (44)$$

where β measures how a credit boom raises the trend growth rate of output, and γ is a small direct effect of credit on the output measurement (a contemporaneous spillover). The state-space matrices are

$$F = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \quad H = (1 \quad 0), \quad B = \begin{pmatrix} 0 \\ \beta \end{pmatrix}, \quad D = \gamma, \quad (45)$$

with $Q = \text{diag}(\varepsilon, \sigma_e^2)$ and $R = \sigma_v^2$. The exogenous loading B injects credit-gap shocks into the slope equation; D allows a direct effect on the observation.

In the stacked formulation, the dynamics matrix G is unchanged from the HP filter case — it depends only on F . What changes is the forcing vector,

$$c_W = \begin{pmatrix} a_0 \\ Bz_1 \\ \vdots \\ Bz_T \end{pmatrix} = \begin{pmatrix} a_0 \\ (0, \beta z_1)^\top \\ \vdots \\ (0, \beta z_T)^\top \end{pmatrix}, \quad (46)$$

and the measurement equation acquires the offset $\mathcal{D}Z = \gamma(z_1, \dots, z_T)^\top$. Conditional on Z , the precision matrix J of the smoothing problem is *exactly* the same block-tridiagonal matrix as in the no-exogenous HP filter; only h shifts to absorb the deterministic forcing. The smoothed trend $\hat{\tau}_{t|T}$ therefore decomposes additively into the no-exogenous HP trend plus the credit-driven contribution.

This is the practical use case the simultaneous formulation handles most cleanly. Coupled with the EM machinery of Section 6, β and γ can themselves be estimated jointly with F, H, Q, R . The demo `demo_hp_credit.m` does exactly that on simulated quarterly output data with a structural break and outlier contamination, and is the flagship example of the package.

The four examples make the same point. Once F, H, Q, R, B, D are identified from the recursive equations, the abstract framework becomes a concrete sparse linear-algebra problem. Different models just have different sparsity patterns in J , and the conditional Gaussian formula of the next subsection applies to all of them uniformly.

3.4 Smoothing as Gaussian conditioning

Apply to (27) the textbook formula for the conditional distribution of one block of a multivariate normal given another. Loosely speaking, what comes out is just a regression — of the entire latent path X on all the observations Y , with the prior on X playing the role of a regularizer. The formula is

$$X \mid Y \sim N(\mu_{X|Y}, \Sigma_{X|Y}) \quad (47)$$

with

$$\mu_{X|Y} = a + \Sigma_X \mathcal{H}^\top (\mathcal{H} \Sigma_X \mathcal{H}^\top + \Sigma)^{-1} (Y - \mathcal{H}a - \mathcal{D}Z), \quad (48)$$

$$\Sigma_{X|Y} = \Sigma_X - \Sigma_X \mathcal{H}^\top (\mathcal{H} \Sigma_X \mathcal{H}^\top + \Sigma)^{-1} \mathcal{H} \Sigma_X. \quad (49)$$

That is the Kalman smoother. There is no recursion, no prediction/update split, no forward sweep. It is exactly the Bayesian GLS regression formula every economist learns in the first semester of graduate econometrics: read $\mathcal{H}$ as the regressor matrix, $Y - \mathcal{D}Z$ as the regressand (the residual after removing the deterministic effect of Z), Σ as the error covariance, and Σ_X^{-1} as the prior precision on X . The smoothed path $\mu_{X|Y}$ is the posterior mean of this regression; $\Sigma_{X|Y}$ is its posterior covariance. Smoothing the latent state is the same kind of object as smoothing the coefficients of a Bayesian regression — it just happens to be a regression on a vector that is very long.

The exogenous input Z enters only via the mean of the regressand, not the covariance, because Z is observed and non-random.

Equations (48)–(49) contain every quantity the smoother produces. Marginal sub-vectors of $\mu_{X|Y}$ are the smoothed states $\hat{x}_{t|T}$. Diagonal blocks of $\Sigma_{X|Y}$ are the smoothed covariances $P_{t|T}$. Adjacent off-diagonal blocks are the lag-one covariances $P_{t,t-1|T}$ that EM requires. The marginal log-likelihood of Y is read off from the marginal Gaussian on the second block, which has mean $\mathcal{H}a + \mathcal{D}Z$ and covariance $\mathcal{H} \Sigma_X \mathcal{H}^\top + \Sigma$.

This is the punch line of the simultaneous formulation: the Kalman smoother is a special case of the multivariate normal conditioning formula. Everything that looks complicated in the recursive derivation comes from refusing to write the joint distribution down.

3.4.1 Where each econometric idea sits

The MVN view turns the rest of this note into a map of standard econometric machinery onto one Gaussian conditioning problem. Five items will reappear later as small edits to either the joint covariance (27) or the precision matrix J defined in the next section.

- **GLS.** Equation (48) is structurally the Bayesian GLS formula: regressor $\mathcal{H}$, regressand $Y - \mathcal{H}a$, error covariance $\mathcal{H} \Sigma_X \mathcal{H}^\top + \Sigma$, prior precision Σ_X^{-1} . Smoothing *is* Bayesian GLS for the regression $Y = \mathcal{H}X + V$ with prior $X \sim N(a, \Sigma_X)$.
- **Marginal log-likelihood.** The marginal distribution of Y in (27) is Gaussian with mean $\mathcal{H}a$ and covariance $\mathcal{H} \Sigma_X \mathcal{H}^\top + \Sigma$, so $\log p(Y; \theta)$ is the concentrated GLS criterion: a quadratic plus a log-determinant. Section 4.6 computes both terms without forming that dense covariance.
- **Missing data.** A missing entry of y_t is one row of $\mathcal{H}$ together with the corresponding row and column of Σ dropped. The joint MVN is still well-defined; the conditional Gaussian still applies. In precision form (Section 4.7), this is one term subtracted from the diagonal block J_{tt} .

- **Diffuse prior.** The $P_0^{-1} \rightarrow 0$ limit corresponds to a flat prior on x_0 , equivalently an unbounded marginal variance in Σ_X . $X | Y$ remains well-defined whenever the observations identify the path. In precision form (Section 4.8), the prior block of J is simply dropped.
- **EM sufficient statistics.** The E-step needs $\mathbb{E}[x_t x'_t | Y]$ and $\mathbb{E}[x_t x'_{t-1} | Y]$. These are diagonal and adjacent off-diagonal blocks of $\Sigma_{X|Y}$ in (49), plus outer products of smoothed means; Section 6 reads them off.

Each item is the same MVN object viewed through a different econometric lens. The next sections introduce the precision form not because it is conceptually different but because it computes these same quantities efficiently on a very long Gaussian.

3.4.2 Why we don't compute estimates directly

Equations (48)–(49) are mathematically perfect, but evaluating them as written — with dense matrix algebra — is computationally wasteful. Three large dense objects appear:

- $\Sigma_X = G^{-1} \Omega G^{-\top}$ is dense, since G^{-1} is lower triangular but full;
- $\mathcal{H} \Sigma_X \mathcal{H}^\top + \Sigma$ is dense of size $mT \times mT$;
- inverting it naively costs $O(m^3 T^3)$.

This is the cost of treating the smoothing problem as a generic regression on a vector of length $(T+1)n$. The next several sections show that the same calculation can be organized to exploit the state-space structure: although $\Sigma_{X|Y}$ is dense, its inverse $J = \Sigma_{X|Y}^{-1}$ is sparse and block tridiagonal. Working with J instead of $\Sigma_{X|Y}$ reduces the cost from $O(T^3 n^3)$ to $O(T n^3)$. No new mathematics is involved — only a change of representation that makes the sparsity visible.

3.5 Identification

Identification is not optional. For any nonsingular M ,

$$\tilde{x}_t = M x_t, \quad \tilde{F} = M F M^{-1}, \quad \tilde{H} = H M^{-1}, \quad \tilde{Q} = M Q M' \quad (50)$$

produce the same observed process. Therefore the latent state is only identified up to invertible transformations unless restrictions are imposed. Common restrictions include:

- observed-state normalizations, for example fixing part of H to $[I \ 0]$;
- factor-model normalizations, such as diagonal or triangular loading matrices;
- setting the scale of the latent shocks, for example $Q = I$;
- imposing a companion form, zero restrictions, or stability restrictions on F ;
- economic long-run restrictions or sign restrictions.

When comparing estimated F, H to true values, compare *invariants* of the latent dynamics (eigenvalues of F , achieved log-likelihood, or the impulse responses of y_t) rather than entries of F and H .

4 The simultaneous smoother in precision form

The remainder of this section develops the precision-form representation of the smoother in full. The flow is: posterior precision and information vector (§4.1); the block-tridiagonal sparsity pattern that makes everything cheap (§4.2); the equivalent penalized-least-squares interpretation (§4.3); the block-LDL[⊤] factorization, which is the Kalman filter in disguise (§4.4); the Takahashi backward sweep for posterior covariances, which is the RTS smoother in disguise (§4.5); the marginal log-likelihood (§4.6); and the two standard practical adaptations that come essentially for free in this representation, missing data (§4.7) and diffuse initialization (§4.8).

Each of these is an immediate corollary of the conditional-Gaussian formula (48)–(49), recast in the representation that exposes the state-space sparsity.

4.1 Posterior precision and information vector

The posterior kernel of $X \mid Y$ is

$$-\frac{1}{2}(GX - c_W)' \Omega^{-1}(GX - c_W) - \frac{1}{2}(Y - \mathcal{H}X - \mathcal{D}Z)' \Sigma^{-1}(Y - \mathcal{H}X - \mathcal{D}Z). \quad (51)$$

Collecting terms in X gives the posterior precision matrix

$$J = G' \Omega^{-1} G + \mathcal{H}' \Sigma^{-1} \mathcal{H} \quad (52)$$

and information vector

$$h = G' \Omega^{-1} c_W + \mathcal{H}' \Sigma^{-1} (Y - \mathcal{D}Z). \quad (53)$$

Therefore the smoothed mean and posterior mode solve

$$J \hat{X} = h. \quad (54)$$

This is the simultaneous Kalman smoother.

Several features of this formula deserve emphasis. The exogenous input Z enters *only* the information vector h and never the precision matrix J . Concretely, the dynamics contribution $G' \Omega^{-1} c_W$ adds the propagated terms $Q^{-1} B z_t$ and $-F' Q^{-1} B z_t$ to adjacent blocks of h , while the measurement contribution replaces Y with the residual $Y - \mathcal{D}Z$ in the right-hand side. The block-tridiagonal sparsity, the LDL[⊤] factorization cost, and the selected-inversion machinery are all identical to the no-exogenous case. Switching exogenous inputs on and off changes only the right-hand side of one sparse linear solve.

Equation (52) is just the Woodbury rewriting of the dense formula (49). To see this, recall that $\Sigma_X^{-1} = G' \Omega^{-1} G$ (since $\Sigma_X = G^{-1} \Omega G^{-\top}$), and apply the matrix inversion lemma to (49):

$$\Sigma_{X|Y}^{-1} = \Sigma_X^{-1} + \mathcal{H}' \Sigma^{-1} \mathcal{H} = G' \Omega^{-1} G + \mathcal{H}' \Sigma^{-1} \mathcal{H} = J. \quad (55)$$

Similarly $\hat{X} = \mu_{X|Y} = J^{-1} h$ reproduces (48). So $J \hat{X} = h$ is not a separate derivation of the smoother; it is the same Gaussian conditioning written in a representation that exposes the sparsity.

4.2 The block-tridiagonal structure

Since G is block bidiagonal, $G' \Omega^{-1} G$ is block tridiagonal. Since measurements enter one period at a time, $\mathcal{H}' \Sigma^{-1} \mathcal{H}$ is block diagonal. Thus J is block tridiagonal.

For the time-invariant model,

$$J = \begin{pmatrix} P_0^{-1} + F'Q^{-1}F & -F'Q^{-1} & 0 & \cdots & 0 \\ -Q^{-1}F & Q^{-1} + F'Q^{-1}F + H'R^{-1}H & -F'Q^{-1} & \cdots & 0 \\ 0 & -Q^{-1}F & Q^{-1} + F'Q^{-1}F + H'R^{-1}H & \ddots & \vdots \\ \vdots & \vdots & \ddots & \ddots & -F'Q^{-1} \\ 0 & 0 & \cdots & -Q^{-1}F & Q^{-1} + H'R^{-1}H \end{pmatrix}. \quad (56)$$

The corresponding right-hand side is, in the no-exogenous case,

$$h = \begin{pmatrix} P_0^{-1}a_0 \\ H'R^{-1}y_1 \\ H'R^{-1}y_2 \\ \vdots \\ H'R^{-1}y_T \end{pmatrix}. \quad (57)$$

With exogenous inputs, the right-hand side becomes

$$h = \begin{pmatrix} P_0^{-1}a_0 - F'Q^{-1}Bz_1 \\ H'R^{-1}(y_1 - Dz_1) + Q^{-1}Bz_1 - F'Q^{-1}Bz_2 \\ H'R^{-1}(y_2 - Dz_2) + Q^{-1}Bz_2 - F'Q^{-1}Bz_3 \\ \vdots \\ H'R^{-1}(y_T - Dz_T) + Q^{-1}Bz_T \end{pmatrix}, \quad (58)$$

where the Bz_t contributions enter from both the predicting and the predicted side of each transition. The block-tridiagonal structure of J is unaffected.

4.3 Equivalent penalized least-squares problem

The same object is obtained as the minimizer of

$$L(X) = (x_0 - a_0)'P_0^{-1}(x_0 - a_0) \quad (59)$$

$$+ \sum_{t=1}^T (x_t - Fx_{t-1} - Bz_t)'Q^{-1}(x_t - Fx_{t-1} - Bz_t) \quad (60)$$

$$+ \sum_{t=1}^T (y_t - Hx_t - Dz_t)'R^{-1}(y_t - Hx_t - Dz_t). \quad (61)$$

The first-order conditions are exactly $J\hat{X} = h$. This view is usually the most transparent one: smoothing is a structured penalized least-squares problem. The exogenous terms Bz_t and Dz_t enter the objective as deterministic shifts of the residuals, so they fall out when the gradient is taken; they appear only on the right-hand side of the resulting linear system.

4.4 The recursive filter as a block-LDL[⊤] forward sweep

The recursive Kalman filter and the simultaneous smoother are the same object. This section makes that statement precise.

Because J is symmetric positive definite and block tridiagonal, it admits a unique block-LDL[⊤] factorization

$$J = LDL^{\top}, \quad (62)$$

where L is block lower-bidiagonal with identity diagonal blocks, and $D = \text{diag}(D_0, D_1, \dots, D_T)$. Write the off-diagonal blocks of L as $L_{t,t-1} = M_t$; this is the convention used by the package code (`simks_selected_inv`).⁴

Matching blocks of $LDL^{\top}$ to blocks of J determines the factors. The $(t, t-1)$ block of $LDL^{\top}$ is $L_{t,t-1}D_{t-1} = M_t D_{t-1}$, and the (t, t) block is $D_t + M_t D_{t-1} M_t^{\top}$. Because the blocks do not commute, the order of multiplication matters: M_t must be recovered by a *right* solve against D_{t-1} ,

$$D_0 = J_{00} = P_0^{-1} + F'Q^{-1}F, \quad (63)$$

$$M_t = J_{t,t-1} D_{t-1}^{-1} = -Q^{-1}F D_{t-1}^{-1}, \quad t \geq 1, \quad (64)$$

$$D_t = J_{tt} - M_t D_{t-1} M_t^{\top}. \quad (65)$$

This recursion is the Kalman filter, written in precision form — but the identification of the pivots requires one step of care. The diagonal pivots D_t are *not* the filtered precisions for interior periods, because the block $J_{tt} = Q^{-1} + F'Q^{-1}F + H'R^{-1}H$ natively carries the prior-precision contribution $F'Q^{-1}F$ of the *outgoing* transition to x_{t+1} . Define instead

$$C_t = D_t - F'Q^{-1}F \quad (t < T), \quad C_T = D_T, \quad (66)$$

stripping the outgoing term (the terminal block has no outgoing transition, which is why D_T needs no correction). Substituting (65) and J_{tt} into (66),

$$C_t = Q^{-1} - Q^{-1}F (C_{t-1} + F'Q^{-1}F)^{-1}F'Q^{-1} + H'R^{-1}H, \quad (67)$$

and the Woodbury identity collapses the first two terms:

$$C_t = (Q + F C_{t-1}^{-1} F')^{-1} + H'R^{-1}H. \quad (68)$$

Setting $P_{t|t} = C_t^{-1}$, this is exactly

$$P_{t|t}^{-1} = P_{t|t-1}^{-1} + H'R^{-1}H, \quad P_{t|t-1} = F P_{t-1|t-1} F' + Q, \quad (69)$$

the filtered/predicted covariance updates of the textbook Kalman recursion, in their precision-form equivalents, with the anchor $C_0 = D_0 - F'Q^{-1}F = P_0^{-1} = P_{0|0}^{-1}$. The forward solve $Lu = h$ likewise produces the filtered *means* in information form: $u_t = C_t \hat{x}_{t|t} = P_{t|t}^{-1} \hat{x}_{t|t}$, the information state of the information-filter literature.

The backward solve $L^{\top} \hat{X} = D^{-1}u$ then propagates information backward. Identifying the smoothing gain

$$A_t = P_{t|t} F' P_{t+1|t}^{-1} \quad (70)$$

in the back-substitution yields

$$\hat{x}_{t|T} = \hat{x}_{t|t} + A_t (\hat{x}_{t+1|T} - F \hat{x}_{t|t}), \quad (71)$$

which is the Rauch–Tung–Striebel smoother.

The message is not that “ $J\hat{X} = h$ is faster than the recursive filter”. Both cost $O(Tn^3)$, because they are the same operation written two ways. The message is that the Kalman filter is the block-LDL[⊤] factorization of J done by hand, and that this factorization is what makes the operation $O(Tn^3)$ rather than $O(T^3n^3)$. Modern sparse direct solvers (CHOLMOD, etc.) discover this factorization automatically. See Davis (2006).

⁴Some references instead set $L_{t,t-1} = -M_t$, which flips the sign of every M_t below and of the off-diagonal line in the Takahashi recursion of Section 4.5. Either convention is fine; mixing them is not.

4.5 Posterior covariances via selected inversion

The posterior covariance of the smoothed path is J^{-1} . Forming J^{-1} densely costs $O(T^3 n^3)$ and is wasteful: we usually want only the diagonal and first sub-diagonal blocks

$$\Sigma_{tt} = \text{Cov}(x_t | Y), \quad \Sigma_{t+1,t} = \text{Cov}(x_{t+1}, x_t | Y), \quad (72)$$

for EM updates and for posterior bands.

These blocks can be computed by Takahashi's recursion, due to [Takahashi et al. \(1973\)](#) and [Erismann and Tinney \(1975\)](#). Starting from $LDL^\top$ of the previous section, $J^{-1} = L^{-\top} D^{-1} L^{-1}$. With $M_t = L_{t,t-1} = J_{t,t-1} D_{t-1}^{-1}$ exactly as defined in Section 4.4, the recursion is, for $t = T, T-1, \dots, 0$,

$$\Sigma_{TT} = D_T^{-1}, \quad (73)$$

$$\Sigma_{t+1,t} = -\Sigma_{t+1,t+1} M_{t+1}, \quad (74)$$

$$\Sigma_{tt} = D_t^{-1} + M_{t+1}^\top \Sigma_{t+1,t+1} M_{t+1}. \quad (75)$$

(These three lines are derived by equating the strict upper triangle of $L^\top \Sigma = D^{-1} L^{-1}$ to its right-hand side and reading off blocks; the sign on the middle line is tied to the convention $L_{t,t-1} = +M_t$ — under the alternative convention $L_{t,t-1} = -M_t$ it flips, a classic source of transcription errors. Note that for the state-space blocks $M_{t+1} = -Q^{-1} F D_t^{-1}$, so $\Sigma_{t+1,t} = \Sigma_{t+1,t+1} Q^{-1} F D_t^{-1}$ is positive in the scalar AR(1) case with $\phi > 0$, as it must be: adjacent smoothed states are positively correlated under persistent dynamics.) That is: at each step backward, we need the diagonal block just above and only the LDL factors of J . Total cost $O(Tn^3)$.

The familiar RTS smoother covariance update

$$P_{t|T} = P_{t|t} + A_t(P_{t+1|T} - P_{t+1|t})A_t' \quad (76)$$

is exactly this Takahashi backward recursion in the precision-form coordinates of Section 4.4.

4.6 Marginal log-likelihood

For maximum likelihood, model comparison, or any kind of likelihood-based inference, we need $\log p(Y; \theta)$, not the joint $\log p(X, Y; \theta)$ at a particular X . The simultaneous formulation gives it cleanly.

From (16) and (22), (X, Y) is jointly Gaussian. Standard manipulations give

$$\log p(Y; \theta) = -\frac{1}{2}(Y - \mathcal{H}a - \mathcal{D}Z)' \Omega_Y^{-1} (Y - \mathcal{H}a - \mathcal{D}Z) - \frac{1}{2} \log \det \Omega_Y - \frac{mT}{2} \log(2\pi), \quad (77)$$

where $a = G^{-1} c_W$ (which now incorporates the propagated Bz_t) and $\Omega_Y = \mathcal{H}G^{-1}\Omega G^{-\top} \mathcal{H}' + \Sigma$. Working directly with Ω_Y is what the recursive Kalman filter does implicitly via the innovations representation, at $O(Tn^3)$ cost.

The simultaneous form is more direct. Complete the square in X :

$$\log p(X, Y; \theta) = -\frac{1}{2} X' J X + h' X + \text{const}(\theta, Y). \quad (78)$$

Integrating X out using $\int \exp(-\frac{1}{2} X' J X + h' X) dX = (2\pi)^{(T+1)n/2} (\det J)^{-1/2} \exp(\frac{1}{2} h' J^{-1} h)$ gives

$$\log p(Y; \theta) = +\frac{1}{2} h' J^{-1} h - \frac{1}{2} \log \det J + \text{const}'(\theta, Y), \quad (79)$$

where the remaining constant collects the prior and measurement normalizing terms. The signs deserve a second look, because both are easy to get backwards: the completed-square quadratic

enters with a *plus* ($+\frac{1}{2}h'J^{-1}h$), and the log-determinant enters with a *minus* ($-\frac{1}{2}\log\det J$). The sanity check on the latter: a more concentrated posterior on the latent path means a larger $\det J$, and concentrating the latent states strictly *reduces* the integrated mass, so $\log\det J$ must lower the marginal likelihood. Writing out the constant, and using $\hat{X} = J^{-1}h$ so that $h'J^{-1}h = \hat{X}'h$,

$$\begin{aligned}\log p(Y; \theta) = & \frac{1}{2}\hat{X}'h - \frac{1}{2}c_W'\Omega^{-1}c_W - \frac{1}{2}(Y - \mathcal{D}Z)'\Sigma^{-1}(Y - \mathcal{D}Z) \\ & - \frac{1}{2}\log\det J - \frac{1}{2}\log\det\Omega - \frac{1}{2}\log\det\Sigma - \frac{mT}{2}\log(2\pi).\end{aligned}\quad (80)$$

Two exogenous-input details matter here and are easy to drop. First, the measurement quadratic uses the exogenous-adjusted residual $Y - \mathcal{D}Z$, not Y itself. Second, the dynamics quadratic uses the full stacked offset $c_W = (a_0', (Bz_1)', \dots, (Bz_T)')'$, so

$$c_W'\Omega^{-1}c_W = a_0'P_0^{-1}a_0 + \sum_{t=1}^T (Bz_t)'Q^{-1}(Bz_t), \quad (81)$$

and h in the $\hat{X}'h$ term is the full information vector of (53), which already carries the propagated Bz_t contributions. (With $B = D = 0$ both reduce to the familiar no-exogenous expressions $a_0'P_0^{-1}a_0$ and $Y'\Sigma^{-1}Y$.) Every term in (80) is cheap. The log-determinant $\log\det J$ is read off from the diagonal of D in $J = LDL^\top$, or equivalently from twice the sum of log of the diagonal of the Cholesky factor. Sparse Cholesky on J delivers both $\hat{X}$ and $\log\det J$ in a single call.

4.7 Missing data

Suppose at time t only a subset $S_t \subseteq \{1, \dots, m\}$ of the observation channels is available, and $y_t^{S_t}$ denotes the observed sub-vector. In the simultaneous formulation, the only thing that changes is the measurement contribution at t :

$$H'R^{-1}H \longrightarrow H'_{S_t}R_{S_t}^{-1}H_{S_t}, \quad H'R^{-1}y_t \longrightarrow H'_{S_t}R_{S_t}^{-1}y_t^{S_t}. \quad (82)$$

A fully missing observation contributes nothing; the corresponding diagonal block of J is just $Q^{-1} + F'Q^{-1}F$ (or its endpoint analogue), with no measurement term.

There is no special “missing-data Kalman filter”. There is just J and h , assembled from whatever measurements exist.

4.8 Diffuse initialization

If we do not want to commit to a prior on x_0 , take the limit $P_0^{-1} \rightarrow 0$. In (54), this simply drops the prior block from the top-left of J and the term $P_0^{-1}a_0$ from h . Provided the dynamics and measurements together identify x_0 (essentially: x_0 appears in some path that has observations downstream), J remains positive definite and the solve is well-posed. This is the “proper” diffuse limit; the recursive analogue requires the careful exact-initial-Kalman-filter treatment of [de Jong \(1991\)](#) and [Koopman and Durbin \(2003\)](#).

5 Time-varying systems

A third small edit, also forecast in Section 3.3. Replacing the constant matrices F, H, Q, R with sequences F_t, H_t, Q_t, R_t leaves the conditional Gaussian (48)–(49) formally unchanged; only the

entries of G , Ω , $\mathcal{H}$, and Σ change. For

$$x_t = F_t x_{t-1} + w_t, \quad w_t \sim N(0, Q_t), \quad (83)$$

$$y_t = H_t x_t + v_t, \quad v_t \sim N(0, R_t), \quad (84)$$

the dynamics matrix becomes

$$G = \begin{pmatrix} I & 0 & 0 & \cdots & 0 \\ -F_1 & I & 0 & \cdots & 0 \\ 0 & -F_2 & I & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & 0 \\ 0 & 0 & \cdots & -F_T & I \end{pmatrix} \quad (85)$$

and $\Omega = \text{diag}(P_0, Q_1, \dots, Q_T)$. The precision matrix remains block tridiagonal, with blocks

$$J_{00} = P_0^{-1} + F_1' Q_1^{-1} F_1, \quad (86)$$

$$J_{tt} = Q_t^{-1} + F_{t+1}' Q_{t+1}^{-1} F_{t+1} + H_t' R_t^{-1} H_t, \quad 1 \leq t \leq T-1, \quad (87)$$

$$J_{TT} = Q_T^{-1} + H_T' R_T^{-1} H_T, \quad (88)$$

$$J_{t,t-1} = -Q_t^{-1} F_t, \quad J_{t-1,t} = -F_t' Q_t^{-1}. \quad (89)$$

Everything downstream — LDL[⊤] factorization, selected inversion, marginal log-likelihood, EM — carries over unchanged. This is the modularity claim in concrete form.

6 Estimating Model Parameters

Suppose F and H are unknown but constant over time. The natural objective is

$$\min_{X, F, H, Q, R} (x_0 - a_0)' P_0^{-1} (x_0 - a_0) \quad (90)$$

$$+ \sum_{t=1}^T (x_t - F x_{t-1})' Q^{-1} (x_t - F x_{t-1}) \quad (91)$$

$$+ \sum_{t=1}^T (y_t - H x_t)' R^{-1} (y_t - H x_t), \quad (92)$$

together with the implicit $-T \log \det Q - T \log \det R$ from the Gaussian normalizing constants if Q, R are also to be estimated.

6.1 Quick-and-dirty: alternating regressions

The simplest scheme is: given (F, H, Q, R) , solve for X with the simultaneous smoother; given X , update F and H by ordinary least squares,

$$\hat{F} = (\sum_t x_t x_{t-1}') (\sum_t x_{t-1} x_{t-1}')^{-1}, \quad \hat{H} = (\sum_t y_t x_t') (\sum_t x_t x_t')^{-1}, \quad (93)$$

and then update Q, R by residual covariances. This is conceptually clean but the Q and R estimates are biased: they ignore the fact that the smoothed x_t is a posterior mean, not a sample from the posterior.

6.2 Proper Gaussian EM

The unbiased fix is the standard EM algorithm of [Shumway and Stoffer \(1982\)](#) and [Watson and Engle \(1983\)](#). The E-step computes the smoothed sufficient statistics

$$S_{00} = \sum_{t=1}^T \mathbb{E}[x_{t-1}x'_{t-1} | Y] = \sum_{t=1}^T (\hat{x}_{t-1|T}\hat{x}'_{t-1|T} + \Sigma_{t-1,t-1}), \quad (94)$$

$$S_{11} = \sum_{t=1}^T \mathbb{E}[x_t x'_t | Y] = \sum_{t=1}^T (\hat{x}_{t|T}\hat{x}'_{t|T} + \Sigma_{tt}), \quad (95)$$

$$S_{10} = \sum_{t=1}^T \mathbb{E}[x_t x'_{t-1} | Y] = \sum_{t=1}^T (\hat{x}_{t|T}\hat{x}'_{t-1|T} + \Sigma_{t,t-1}). \quad (96)$$

The posterior covariance terms Σ_{tt} and $\Sigma_{t,t-1}$ are delivered by the selected-inversion recursion of Section 4.5. The M-step then solves

$$\hat{F} = S_{10}S_{00}^{-1}, \quad (97)$$

$$\hat{Q} = \frac{1}{T}(S_{11} - S_{10}S_{00}^{-1}S'_{10}), \quad (98)$$

$$\hat{H} = (\sum_t y_t \hat{x}'_{t|T})(\sum_t (\hat{x}_{t|T}\hat{x}'_{t|T} + \Sigma_{tt}))^{-1}, \quad (99)$$

$$\hat{R} = \frac{1}{T} \sum_t ((y_t - \hat{H}\hat{x}_{t|T})(y_t - \hat{H}\hat{x}_{t|T})' + \hat{H}\Sigma_{tt}\hat{H}'). \quad (100)$$

The initial-state parameters are updated as well, by the standard [Shumway and Stoffer \(1982\)](#) single-observation M-step

$$\hat{a}_0 = \hat{x}_{0|T}, \quad \hat{P}_0 = \Sigma_{00} (= \Sigma_{00} + (\hat{x}_{0|T} - \hat{a}_0)(\hat{x}_{0|T} - \hat{a}_0)' \text{ evaluated at the new } \hat{a}_0), \quad (101)$$

which is what `simks_em_const` and `em_const` implement; the returned `a0` and `P0` are therefore genuinely estimated quantities, not pass-throughs of the initialization. (Estimating (a_0, P_0) from one effective observation is, of course, weakly informative; fixing them, or using the diffuse initialization of Section 4.8, are the standard alternatives.) The marginal log-likelihood (80) is non-decreasing across iterations. Monitoring it is a useful sanity check; the demo `demo_const_em.m` does exactly that.

The cost per iteration is dominated by the smoother solve and the selected inversion, both $O(Tn^3)$.

6.3 Exogenous loadings as a regression augmentation

When the model includes exogenous inputs, the EM M-step generalizes in the most natural way possible. The state-side update becomes a joint regression of $\hat{x}_{t|T}$ on the augmented regressor $(\hat{x}_{t-1|T}, z_t)$:

$$\begin{pmatrix} \hat{F} & \hat{B} \end{pmatrix} = (S_{10} \ S_{1z}) \begin{pmatrix} S_{00} & S_{0z} \\ S_{0z}^\top & S_{zz} \end{pmatrix}^{-1}, \quad (102)$$

where $S_{1z} = \sum_t \hat{x}_{t|T} z_t^\top$, $S_{0z} = \sum_t \hat{x}_{t-1|T} z_t^\top$, and $S_{zz} = \sum_t z_t z_t^\top$. The same construction applies to the measurement side:

$$\begin{pmatrix} \hat{H} & \hat{D} \end{pmatrix} = (S_{yx} \ S_{yz}) \begin{pmatrix} S_{xx} & S_{xz} \\ S_{xx}^\top & S_{zz}^{\text{obs}} \end{pmatrix}^{-1}, \quad (103)$$

and the noise covariances $\widehat{Q}, \widehat{R}$ then follow by the standard residual-sum-of-squares formulas with cross-terms.

The shape of these updates is the substantive payoff of including z_t from the outset. Estimating the loading of a credit gap on output trend is mechanically identical to estimating the autoregressive coefficient: both are coefficients on a regressor in $\widehat{x}_t = [\widehat{F} \widehat{B}](\widehat{x}_{t-1}, z_t)^\top$. The simultaneous smoother delivers the smoothed sufficient statistics the regression needs.

The demo `demo_hp_credit.m` runs this machinery on a simulated HP-plus-credit-cycle setup with structural breaks and outlier contamination. To be precise about what is combined and how: the demo compares *separate* estimators on the same data — Gaussian smoothing without and with the true exogenous loadings, robust Student- t smoothing (Section 7.1) at the true parameters, and Gaussian EM that estimates (F, H, Q, R, B, D) jointly. It does *not* run an integrated robust-EM procedure (alternating IRLS weight updates with M-step parameter updates); such a generalized EM is a natural and straightforward extension of the two building blocks, but it is not implemented in the package, and we flag that explicitly rather than imply otherwise. The realistic ingredients for applied macroeconomic work are nonetheless all present: parameters are unknown, the data are imperfect, and the underlying dynamics include observed drivers.

7 Beyond the Standard Model

So far the smoothing problem has been a textbook linear Gaussian state-space system with constant or time-varying matrices. Most substantively interesting macroeconomic state-space problems violate at least one of these assumptions: data contain outliers or structural breaks (so the Gaussian assumption fails); regimes shift between expansions and recessions (so the linearity assumption fails); or known structural bounds rule out parts of the latent path (so the posterior is truncated). The recursive Kalman filter loses its closed-form structure in each of these cases and is patched only by case-specific machinery. The simultaneous form, by contrast, handles all three with the same sparse linear-algebra primitives: in every case the algorithm is either iteratively reweighted solves on the same J , or variational EM with J assembled from responsibility-weighted expected quadratic forms, or a sparse QP whose Hessian is J plus a low-rank update. This section develops the three cases.

7.1 Non-Gaussian disturbances

The stacked formulation extends naturally beyond Gaussian errors. Replace the quadratic penalties by general negative log densities:

$$\min_X \sum_{t=1}^T \rho_w(x_t - Fx_{t-1}) + \sum_{t=1}^T \rho_v(y_t - Hx_t) + \rho_0(x_0 - a_0). \quad (104)$$

The exact closed-form solve is lost, but the sparsity is preserved: each x_t still interacts only with x_{t-1} , x_{t+1} , and y_t . The same J pattern, with weights updated each iteration, supports a broad family of robust state-space smoothers. This section makes that concrete.

7.1.1 Heavy tails as Gaussian scale mixtures

The practically important case is heavy-tailed measurement noise (to robustify against outliers) and heavy-tailed state innovations (to permit structural breaks). Both are handled cleanly by the

Gaussian scale mixture representation (Andrews and Mallows, 1974):

$$v_t \mid \tau_t \sim N(0, R/\tau_t), \quad \tau_t \sim p(\tau \mid \text{family}). \quad (105)$$

Marginalizing τ_t out gives the desired heavy-tailed marginal:

- Student- t with ν degrees of freedom: $\tau \sim \text{Gamma}(\nu/2, \nu/2)$.
- Laplace (double exponential): τ has an inverse-exponential distribution.

Throughout this section, $\delta_t = (r_t^\top R^{-1} r_t)^{1/2}$ denotes the *Mahalanobis distance* of the period- t residual, so δ_t^2 is the squared distance (the quadratic form itself); the two are easy to confuse in implementation, and the update formulas below are stated in terms of δ_t^2 where the quadratic form is meant. The Huber loss is deliberately *not* on the list above: its working weight $\min(1, c/\delta_t)$ depends on the realized residual, whereas a mixing distribution $p(\tau)$ in a proper Gaussian scale mixture cannot. Huber smoothing is therefore best understood not as a latent-scale model but as classical IRLS on the Huber ρ -function (Holland and Welsch, 1977): the weight $\psi(\delta)/\delta = \min(1, c/\delta)$ is the majorize–minimize multiplier that makes each reweighted Gaussian solve a tangent quadratic to the Huber objective. It plugs into exactly the same sparse machinery, but its justification is the loss function, not a mixture representation. Conditional on the τ_t , the model is Gaussian with time-varying covariance $R_t = R/\tau_t$, and the simultaneous smoother of Section 5 applies without modification.

7.1.2 EM and IRLS on the same sparse system

The natural inference algorithm alternates reweighted Gaussian solves with weight updates (Lange et al., 1989).

1. **Reweighted solve.** Given the current weights $\tau_t^{(k)}$, solve the time-varying Gaussian smoother with $R_t = R/\tau_t^{(k)}$. This is one sparse Cholesky on the same J pattern.
2. **Weight update.** Given the smoothed $\hat{X}^{(k+1)}$, update each weight from the smoothed residual

$$r_t = y_t - H\hat{x}_{t|T}^{(k+1)} - Dz_t, \quad (106)$$

where the exogenous offset Dz_t must be subtracted whenever the model has measurement-side inputs. For Student- t ,

$$\tau_t^{(k+1)} = \frac{\nu + m_t}{\nu + r_t^\top R^{-1} r_t}, \quad (107)$$

where m_t is the number of observed entries of y_t . For Laplace, $\tau_t = (r_t^\top R^{-1} r_t)^{-1/2}$. For Huber, $\tau_t = \min(1, c/\delta_t)$, with the canonical $c = 1.345$ (Holland and Welsch, 1977).

Each iteration costs one $O(Tn^3)$ sparse Cholesky. The sparsity pattern of J is identical to the Gaussian case; only the numerical values of the diagonal blocks change.

It is worth being precise about what this iteration computes, because two related but distinct targets are in circulation. As written — with weights evaluated by *plugging in* the residual at the current smoothed mean — the fixed point is the posterior *mode* of the heavy-tailed model: plug-in IRLS is exactly majorize–minimize on the non-Gaussian MAP objective. This is a perfectly respectable estimand, and it is the package default. It is *not*, however, the E-step of an exact latent-scale EM algorithm, which requires the posterior *expectation* of the residual quadratic form rather

than its plug-in value. Under the Gaussian approximation to the state posterior, that expectation is the plug-in value plus a covariance correction,

$$\mathbb{E}[r_t^\top R^{-1} r_t | Y] = \hat{r}_t^\top R^{-1} \hat{r}_t + \text{tr}(R^{-1} H \Sigma_{tt} H'), \quad (108)$$

with Σ_{tt} delivered by the selected inversion of Section 4.5. For Student- t the package implements this covariance-corrected E-step as an option (`opts.exact_estep` in Octave, `exact_estep=True` in Python), at the cost of one selected inversion per iteration; the corrected weights are smaller in periods of high state uncertainty, which is the honest behaviour — an apparently clean residual at an imprecisely estimated state is weaker evidence against an outlier than the plug-in update assumes.

7.1.3 State-side robustness

The same construction applies to the state innovations w_t . Heavy tails on w_t accommodate occasional large jumps in the latent path while still smoothing tightly through quiet periods. The IRLS weights ξ_t on w_t are updated by exactly the same formulas as τ_t , using the squared Mahalanobis distance (the quadratic form $\hat{w}_t^\top Q^{-1} \hat{w}_t$, not its square root) of the smoothed innovation

$$\hat{w}_t = \hat{x}_{t|T} - F \hat{x}_{t-1|T} - B z_t, \quad (109)$$

and the time-varying state covariance becomes $Q_t = Q/\xi_t$. The exogenous forcing $B z_t$ must be subtracted here for the same reason as on the measurement side: otherwise known exogenous movements in the state would be misread as structural breaks. The covariance-corrected E-step analogue of (108) uses

$$\mathbb{E}[w_t^\top Q^{-1} w_t | Y] = \hat{w}_t^\top Q^{-1} \hat{w}_t + \text{tr}\left(Q^{-1} [\Sigma_{tt} - F \Sigma'_{t,t-1} - \Sigma_{t,t-1} F' + F \Sigma_{t-1,t-1} F']\right), \quad (110)$$

which requires the lag-one block $\Sigma_{t,t-1}$ in addition to the diagonal blocks — both already produced by the same Takahashi sweep.

This is the practical machinery for structural-break detection. Set the state-innovation prior to Student- t with $\nu = 4$, run the smoother, and read off the periods at which $\xi_t \ll 1$: those are the breaks. The demo `demo_robust_breaks.m` does this on a near-random-walk series with three injected level shifts; the algorithm identifies all three at $\xi_t < 0.02$ while $\xi_t \approx 1$ elsewhere, and the smoothed path achieves under a third of the RMSE of the Gaussian smoother.

7.1.4 Implementation

The package implements all of the above in `simks_smooth_robust.m`. The interface is

```
[Xhat, info] = simks_smooth_robust(Y, F, H, Q, R, a0, P0, family, opts);
```

with `family` one of `'gaussian'`, `'student-t'`, `'laplace'`, `'huber'`, and `opts.robust_state` optionally activating the state-side variant. The returned `info.tau_v` (and `info.tau_w`) is the per-period weight vector at convergence; values much smaller than 1 are diagnostic flags for outlier observations or structural breaks.

The demo `demo_robust.m` contaminates 10% of observations with extra noise and compares the Gaussian, Student- t , Huber, and Laplace smoothers. Student- t and Huber cut RMSE by roughly 40 percent versus Gaussian and correctly downweight most outlier observations.

7.1.5 Beyond scale mixtures

The IRLS-on-the-same- J scheme covers more than Gaussian scale mixtures. Asymmetric Laplace gives quantile-style smoothing; smooth approximations of the ℓ_1 penalty give pseudo-Huber; generalized linear measurement models (Poisson, binomial, multinomial) give non-Gaussian observation links, with the IRLS weights determined by the local curvature of the link function. All of these inherit the sparsity structure of J automatically.

Thus the simultaneous view is not merely an alternative derivation of the Kalman smoother. It is the right abstraction for a broader class of sparse latent-path estimation problems; see [Rue and Held \(2005\)](#) for the spatial-statistics counterpart.

7.2 Regime-switching dynamics

Macroeconomic models often need state-space dynamics that depend on a discrete unobserved regime — expansion versus recession, dovish versus hawkish monetary policy, high versus low financial-stress periods. The natural model is a switching linear dynamical system (SLDS):

$$x_t = F_{s_t} x_{t-1} + B_{s_t} z_t + w_t, \quad w_t \sim N(0, Q_{s_t}), \quad (111)$$

$$y_t = Hx_t + Dz_t + v_t, \quad v_t \sim N(0, R), \quad (112)$$

with $s_t \in \{1, \dots, K\}$ a Markov chain on K regimes with transition matrix A .

The exact posterior $p(X, S \mid Y)$ is a Gaussian mixture with K^T components and intractable for any non-trivial T . The standard scalable approach, due to [Ghahramani and Hinton \(2000\)](#), is variational EM with a single-Gaussian-times-Markov-chain factorization:

$$q(X, S) = q(X) \prod_{t=1}^T q(s_t). \quad (113)$$

This decouples the two latent variables and turns inference into alternating updates of two tractable problems.

7.2.1 Variational EM iteration

Let $\pi_{t,k} = q(s_t = k)$ be the regime responsibilities. Coordinate ascent on the evidence lower bound alternates two exact updates, and it is worth being careful about what “exact” means here, because a tempting shortcut — smoothing under the responsibility-*averaged* matrices $\tilde{F}_t = \sum_k \pi_{t,k} F_k$, $\tilde{Q}_t = \sum_k \pi_{t,k} Q_k$ — is a moment-matching approximation, *not* the variational update, and the two differ whenever the regimes are uncertain (note in particular that $\sum_k \pi_{t,k} Q_k^{-1} \neq (\sum_k \pi_{t,k} Q_k)^{-1}$). The package implements the exact updates.

1. **$q(X)$ -step (smooth under expected quadratic forms).** The optimal coordinate update sets $\log q(X) = \mathbb{E}_{q(S)} \log p(X, S, Y) + \text{const}$, which is Gaussian, with the expected transition term at t averaging the per-regime quadratic forms. Collecting terms in X , the period- t contributions to the precision matrix are

$$\overline{Q^{-1}}_t = \sum_{k=1}^K \pi_{t,k} Q_k^{-1}, \quad \overline{Q^{-1}F}_t = \sum_{k=1}^K \pi_{t,k} Q_k^{-1} F_k, \quad \overline{F'Q^{-1}F}_t = \sum_{k=1}^K \pi_{t,k} F_k' Q_k^{-1} F_k, \quad (114)$$

assembled as $J_{tt} = \overline{Q^{-1}}_t + \overline{F'Q^{-1}F}_{t+1} + H'R^{-1}H$ and $J_{t,t-1} = -\overline{Q^{-1}F}_t$ (with the obvious end-point modifications), and with the information vector carrying the matching mixed exogenous

terms $\overline{Q^{-1}B_t} z_t$ and $-\overline{F'Q^{-1}B_{t+1}} z_{t+1}$. Crucially, this precision matrix is *still block tridiagonal at every iteration* — the expected quadratic forms couple only adjacent periods, exactly as in the fixed-regime model — so each $q(X)$ -step is one sparse Cholesky of the usual structure; only the block values change.

2. **$q(S)$ -step (regime update under expected log-likelihoods).** The optimal $q(s_t)$ update requires the *expected* Gaussian transition log-likelihood under $q(X)$, not the density evaluated at the smoothed point path. Writing $\hat{d}_{t,k} = \hat{x}_t - F_k \hat{x}_{t-1} - B_k z_t$,

$$\begin{aligned} \ell_{t,k} &= \mathbb{E}_{q(X)} \log N(x_t; F_k x_{t-1} + B_k z_t, Q_k) \\ &= -\frac{1}{2} \left[\hat{d}_{t,k}^\top Q_k^{-1} \hat{d}_{t,k} + \text{tr}(Q_k^{-1} \mathcal{C}_{t,k}) \right] - \frac{1}{2} \log \det Q_k - \frac{n}{2} \log 2\pi, \end{aligned} \quad (115)$$

with the covariance correction

$$\mathcal{C}_{t,k} = \Sigma_{tt} - F_k \Sigma'_{t,t-1} - \Sigma_{t,t-1} F'_k + F_k \Sigma_{t-1,t-1} F'_k \quad (116)$$

built from the diagonal and lag-one blocks that the selected inversion of Section 4.5 already produces. The standard discrete forward-backward HMM smoother is then run on the pair (A, e^ℓ) , at $O(TK^2)$ cost, dominated by the state smoothing.

The two steps are iterated to convergence in π . Typically 10–40 iterations suffice for moderate- T macroeconomic data. The covariance corrections matter in both directions: omitting the trace terms (i.e. scoring regimes at the point path) systematically overstates regime-classification confidence, because a residual that looks small at an imprecisely estimated state is weaker evidence than the same residual at a well-pinned-down state. A corollary worth knowing when designing a regime-switching exercise: regimes that differ mainly in innovation *variance* are sharply classifiable only when the posterior state uncertainty is small relative to the quiet regime’s innovation variance — otherwise even an oracle observing the true states classifies imperfectly.

The substantive payoff is that the precision-form formulation extends to regime-switching dynamics without any new mathematical machinery: every $q(X)$ -step is one sparse Cholesky on a block-tridiagonal J of the same pattern the no-switching smoother uses, and the covariance blocks the $q(S)$ -step needs come from the same Takahashi sweep that EM already requires. The recursive form, by contrast, loses its closed-form structure once regimes are introduced — exact filtering would require maintaining K^t Gaussians at time t , with the Kim and Hamilton filters performing per-step collapses to a single Gaussian per regime. The simultaneous form makes a single-Gaussian approximation too, but as the optimal $q(X)$ within the variational family, with a monotone evidence-lower-bound interpretation rather than an ad hoc collapse.

The demo `demo.slds.m` implements a two-regime AR(1) with expansion/recession dynamics (persistence $\phi_1 = 0.85$, $\phi_2 = 0.50$; volatility $\sigma_1^2 = 0.02$, $\sigma_2^2 = 0.20$; measurement noise $R = 0.01$, small relative to σ_1^2 per the corollary above). The demo generates its data with a small portable random-number generator (a Park–Miller linear congruential generator plus Box–Muller, implemented identically in the Octave/MATLAB and Python versions), so all three environments simulate the same data and print the same numbers: regime classification accuracy ≈ 0.90 , mean $\pi_{t,\text{rec}} \approx 0.80$ during true recessions and ≈ 0.06 during true expansions, and SLDS state RMSE of 0.095 against 0.099 for the single-regime Gaussian smoother run at averaged dynamics.

7.3 Constrained smoothing

Many macroeconomic state-space objects have known structural bounds that the textbook Gaussian smoother ignores. A NAIRU estimate that goes negative is meaningless; a debt-to-GDP latent

component that exceeds historical maxima is implausible; an equilibrium real interest rate is sometimes assumed monotone over a regime. These are inequality constraints on the smoothed path,

$$A_{\text{ineq}} X \leq b_{\text{ineq}}. \quad (117)$$

Such constraints have no clean recursive-filter counterpart; the Kalman recursion produces a Gaussian conditional, which is in general not respected by any non-trivial inequality.

7.3.1 Constrained smoothing as a sparse QP

In the simultaneous formulation, constrained smoothing is the quadratic program

$$\min_X \frac{1}{2} X' J X - h' X \quad \text{s.t.} \quad A_{\text{ineq}} X \leq b_{\text{ineq}}, \quad (118)$$

with J and h exactly as before. When the constraints are inactive at the unconstrained optimum, $\hat{X} = J^{-1}h$ already solves (118); when some constraints bind, the solution is the constrained posterior mode.

7.3.2 Interior-point algorithm on the same sparse J

The standard primal-dual interior-point method uses a log-barrier:

$$X(\mu) = \arg \min_X \frac{1}{2} X' J X - h' X - \mu \sum_{i=1}^c \log(b_i - a_i' X). \quad (119)$$

The Newton step at X requires solving

$$\left(J + \mu A_{\text{ineq}}^\top \text{diag}(1/(b - A_{\text{ineq}} X)^2) A_{\text{ineq}} \right) \Delta X = -\nabla, \quad (120)$$

where the gradient is $\nabla = JX - h + \mu A_{\text{ineq}}^\top (1/(b - A_{\text{ineq}} X))$. The sign of the barrier term is worth a sanity check, since the displayed Hessian only pins it down implicitly: differentiating $-\mu \log(b_i - a_i' X)$ gives $+\mu a_i/(b_i - a_i' X)$, a force that blows up as the iterate approaches the boundary and pushes the minimizer back into the interior, exactly as a barrier should; with the opposite sign the “barrier” would attract iterates to the boundary and the method would degenerate into clipping.

The structural point is that the Hessian is J plus a low-rank positive-semidefinite update. For typical macroeconomic constraints — bounds on a single state coordinate at each t , or on adjacent differences — A_{ineq} is itself extremely sparse, often with only $O(n)$ nonzeros per row. The augmented Hessian $J + \mu A_{\text{ineq}}^\top D A_{\text{ineq}}$ then remains block tridiagonal (component-wise bounds) or grows the bandwidth by one block (monotonicity constraints). Either way, sparse Cholesky still applies, and the per-iteration cost stays $O(Tn^3)$.

The barrier parameter μ is decreased geometrically until the duality gap is below tolerance. Total cost is the number of Newton iterations times the sparse-Cholesky cost; typical convergence is 20–60 inner iterations across 30–40 barrier reductions for a moderate-sized state-space problem with component-wise bounds.

7.3.3 Posterior interpretation

The QP solution is the constrained posterior *mode*, not the posterior mean. Under truncated Gaussian posteriors the mean differs from the mode in general; computing the mean requires sampling methods such as Botev (2017)’s exact truncated-Gaussian algorithm. For most applied

macro uses — displaying a smoothed trend that respects non-negativity, or a credible band whose endpoints respect a known structural bound — the mode is the quantity of interest.

The demo `demo_constrained.m` imposes $x_t \geq 0$ on a noisy near-zero AR(1) smoother (a stand-in for a NAIRU-like object) and verifies that the constraint is exactly satisfied while the unconstrained smoother places 31/120 periods below zero. RMSE against the true non-negative trend improves by $\sim 4\%$, illustrating that imposing known structural information is information-efficient, not just cosmetic.

7.3.4 Inequality constraints beyond bounds

The same QP machinery handles a broad set of constraints:

- **Box constraints.** $\ell_t \leq x_{t,j} \leq u_t$ on selected coordinates and periods: two rows per constraint.
- **Monotonicity.** $x_{t+1,j} \geq x_{t,j}$ for a trend component: rows that pick out $-x_{t+1,j} + x_{t,j}$.
- **Smoothness / curvature.** Bounds on $\Delta^2 x_t = x_t - 2x_{t-1} + x_{t-2}$: three-period stencils.
- **Cone constraints.** Bounds on $\|x_t\|_\infty$ etc., representable as collections of linear inequalities.

The simultaneous form turns each into a few extra sparse rows of A_{ineq} , with no impact on the underlying J . This is the kind of extension that is essentially impossible to graft onto a recursive Kalman filter and easy to add here.

8 What you gain

Beyond working in the right idiom, the simultaneous formulation has four concrete advantages over the recursive form.

1. **Numerical robustness, properly scoped.** Sparse Cholesky on J maintains symmetry and positive definiteness by construction: J is assembled, not propagated, so the failure mode of a hand-coded recursive filter — losing definiteness through subtractions of large covariance matrices, the reason the square-root, UDU, and Bierman–Thornton variants exist — cannot occur. The honest counterpart, spelled out in Section 2.1, is that the precision form is a normal-equations representation whose conditioning degrades quadratically where square-root forms degrade linearly, and which needs SPD Q and R (or explicit regularization). For well-posed batch smoothing problems — the use cases of this note — the Cholesky route is robust and simple; for genuinely ill-conditioned online problems the square-root literature retains its documented advantages.
2. **Modularity.** Missing data, diffuse priors, irregular sampling, partial observations, time-varying parameters, and unobserved-components decompositions are all small edits to the assembly of J and h . No need to redesign a recursion or re-derive boundary conditions.
3. **Extensibility.** Robust losses (Laplace, Student- t), sparsity priors, inequality constraints, and hierarchical extensions all plug into the same sparse linear-algebra framework, typically via iteratively reweighted solves on the same J .
4. **Conceptual unification.** The Hodrick–Prescott filter, the Beveridge–Nelson decomposition, unobserved-components models, factor models, and the textbook state-space problem are all instances of a single object: a quadratic penalty on a latent path solved by $J\hat{X} = h$. This unification is hard to see through the recursive lens.

What you do not gain is raw speed. Sparse Cholesky on a block-tridiagonal SPD matrix *is* the block-LDL[⊤] recursion; CHOLMOD finds the natural ordering for free, because the time ordering is already optimal. Both methods are $O(Tn^3)$ and the constants are comparable. The point of the simultaneous form is not that it computes the answer faster. It is that it computes the answer in a form the student can recognize as the answer.

9 Conclusion

The recursive Kalman filter and the RTS smoother are not separate theories. They are the block-LDL[⊤] forward sweep and the selected-inversion backward sweep of one sparse block-tridiagonal linear system,

$$J\hat{X} = h.$$

The reason this is not how they are usually taught is historical: the recursive form was forced on Kalman and his contemporaries by 1960s memory constraints and shaped by an engineering rather than an econometric tradition. Neither constraint is ours. For batch state-space estimation in modern macroeconomics, the simultaneous form is in the native idiom; numerically robust for well-posed problems with SPD precision, with none of the subtraction-induced failure modes of hand-coded recursions; modular under missing data and diffuse priors; easily extended to non-Gaussian errors and unobserved-components decompositions; and at least as efficient. Posterior covariances, marginal likelihood, and EM with smoothed sufficient statistics all follow from the same sparse solve.

License

Copyright © 2026 Mico Mrkaic. The package is released under the GNU General Public License version 3 or later (<https://www.gnu.org/licenses/gpl-3.0.html>). In plain terms: you may use, modify, and redistribute the code freely, including for commercial purposes, provided that derivative works are released under the same license with source code available. Every source file in the package carries a short GPLv3 header pointing to the full license text in the `LICENSE` file at the package root.

A Octave reference implementation

The reference implementation lives in `octave/simksmoothe/` and is written in pure Octave (also runs on MATLAB). It assembles J and h directly as sparse matrices via triplet arrays (i, j, v) with one terminal call to `sparse(i,j,v,N,N)`. No matter which routine is invoked, the fundamental operation is one sparse solve:

```
Xvec = J \ h;
```

Octave's backslash dispatches to CHOLMOD when J is sparse and SPD, so the cost is $O(Tn^3)$ in both flops and memory, just as in the recursive Kalman filter.

A.1 Calling conventions

All routines share the same calling style. Required arguments (`Y`, `F`, `H`, `Q`, `R`, `a0`, `P0`) come first; optional behaviour is controlled through an `opts` struct passed as the final argument. Common fields:

- `opts.Z` : $k \times T$ exogenous inputs. Omit for none.
- `opts.B` : $n \times k$ state-side loading (default zeros).
- `opts.D` : $m \times k$ measurement-side loading (default zeros).
- `opts.return_cov` : if `true`, also return posterior covariance blocks via selected inversion.

Diffuse priors are signalled by `P0 = []`. Missing observations are NaN entries in `Y`; they are dropped from $\mathcal{H}^\top R^{-1} \mathcal{H}$ at the affected periods automatically. The path `Xhat` returned by every routine is $n \times (T + 1)$, with columns $\hat{x}_{0|T}, \dots, \hat{x}_{T|T}$.

A.2 Core smoothing and inference

`simks_smooth_const`. Basic constant-parameter smoother; the workhorse.

```
[Xhat, J, h, info] = simks_smooth_const(Y, F, H, Q, R, a0, P0, opts)
```

Inputs: the seven required state-space arguments plus the optional `opts` struct. *Outputs*: the smoothed path `Xhat`; the sparse precision matrix `J`; the information vector `h`; and an `info` struct that always contains n , m , T and the nonzero count of J , and additionally `Ptt`, `Ptt1` (cell arrays of diagonal and first-subdiagonal blocks of J^{-1}) when `opts.return_cov=true`.

`simks_smooth_tv.` Time-varying smoother.

```
[Xhat, J, h, info] = simks_smooth_tv(Y, Fseq, Hseq, Qseq, Rseq, a0, P0, opts)
```

Each of `Fseq`, `Hseq`, `Qseq`, `Rseq` may be a single constant matrix (replicated across all t) or a $1 \times T$ cell array of per-period matrices; mixed usage is supported. Outputs as in `simks_smooth_const`.

`simks_selected_inv.` Takahashi selected inversion.

```
[Ptt, Ptt1, logdetJ] = simks_selected_inv(J, n)
```

Inputs: the precision matrix J and the block size n . *Outputs:* the diagonal blocks Σ_{tt} and the first-subdiagonal blocks $\Sigma_{t,t-1}$ of J^{-1} as cell arrays, plus $\log \det J$ as a byproduct. These are precisely the sufficient statistics the EM E-step needs.

`simks_marginal_loglik.` Marginal log-likelihood.

```
[ll, parts] = simks_marginal_loglik(Y, F, H, Q, R, a0, P0, opts)
```

Returns the scalar marginal log-likelihood $\log p(Y; \theta)$ via the formula in Section 4.6. The `parts` struct breaks out the prior, data, $\log \det J$, and normalizing contributions for diagnostics.

`simks_simulate_const.` Forward simulation.

```
[Y, X] = simks_simulate_const(T, F, H, Q, R, a0, P0, opts)
```

Inputs: the horizon T and the state-space parameters. *Outputs:* Y ($m \times T$) and X ($n \times (T + 1)$), columns $x_0, \dots, x_T$. Used by the demos and the test suite to generate ground-truth data.

`simks_rts_reference.` Textbook Kalman + RTS.

```
[Xhat, P] = simks_rts_reference(Y, F, H, Q, R, a0, P0)
```

A clean recursive implementation, provided so that the reader can compare the recursive and simultaneous formulations line by line and confirm they compute the same object to machine precision. No exogenous-input or missing-data handling — it is the textbook algorithm in its simplest form.

A.3 Parameter estimation

`simks_em_const.` Gaussian EM.

```
[params, Xhat, hist] = simks_em_const(Y, n, opts)
```

Inputs: the data Y , the latent state dimension n , and an `opts` struct that may carry initial parameter values, convergence controls, and exogenous inputs Z . *Outputs:* the estimated parameter struct `params` with fields `F`, `H`, `Q`, `R`, `a0`, `P0` (plus `B`, `D` if exogenous was supplied); the final smoothed path; and the history of the marginal log-likelihood across iterations. The history is monotone non-decreasing by construction and is a useful sanity check. All returned parameters, including `a0` and `P0`, are estimated: each iteration updates the initial-state pair by the single-observation M-step $\hat{a}_0 = \hat{x}_{0|T}$ and $\hat{P}_0 = \Sigma_{00}$ documented in Section 6.

A.4 Beyond-Gaussian and constrained smoothing

`simks_smooth_robust`. Heavy-tailed IRLS smoother.

```
[Xhat, info] = simks_smooth_robust(Y, F, H, Q, R, a0, P0, family, opts)
```

Inputs: the standard state-space arguments plus the `family` string and `opts`. The `family` is one of 'gaussian', 'student-t', 'laplace', or 'huber'. The `opts` struct may carry `nu` (Student- t degrees of freedom), `c` (Huber tuning constant), `robust_state` (boolean, applies heavy tails to w_t as well), `exact_estep` (boolean, Student- t only: replaces plug-in IRLS with the covariance-corrected E-step of Section 7.1), and the usual exogenous fields. *Outputs:* the smoothed path and an `info` struct that always contains `tau_v` (per-period measurement weights at convergence), and additionally `tau_w` when `robust_state` is true. Weights much below 1 flag outliers or structural breaks.

`simks_smooth_slds`. Switching dynamics.

```
[Xhat, info] = simks_smooth_slds(Y, F_set, H, Q_set, R, a0, P0, opts)
```

Variational EM (exact coordinate updates, Section 7.2) for the switching linear dynamical system. *Inputs:* `F_set` and `Q_set` are $1 \times K$ cell arrays of per-regime matrices; `opts.A` is the $K \times K$ regime-transition matrix; with exogenous inputs, the per-regime state loadings are passed as `opts.B_set`, a $1 \times K$ cell array of $n \times k$ matrices (defaulting to zeros), alongside the common `opts.Z` and the regime-invariant `opts.D` — note that the `opts.B` field of `simks_smooth_const` is *not* used here, precisely because the loading is regime-dependent. The remaining arguments are as in `simks_smooth_const`. *Outputs:* the smoothed state path; `info.pi`, a $K \times T$ matrix of regime responsibilities $\pi_{t,k}$; and `info.Ptt`, `info.Ptt1`, the posterior covariance blocks of $q(X)$ at convergence (used internally by the expected-log-likelihood regime update and returned for convenience).

`simks_smooth_constrained`. Inequality-constrained smoother.

```
[Xhat, info] = simks_smooth_constrained(Y, F, H, Q, R, a0, P0, A_ineq, b_ineq, opts)
```

Inputs: the standard state-space arguments plus the constraint data A_{ineq} (sparse, $c \times (T + 1)n$) and b_{ineq} (length c). *Outputs:* the constrained smoothed path `Xhat` and an `info` struct containing the active-set mask, the approximate KKT multipliers, and interior-point convergence diagnostics.

A.5 Demos

The `octave/examples/` directory contains eleven demos exercising every routine and feature of the package. Every demo prints numerical diagnostics to the console and produces a figure showing the relevant smoothed trajectories, posterior bands, IRLS weights, or regime probabilities. Figures are simultaneously saved to `octave/examples/figures/` and displayed on screen. The three worked applications in Appendix C produce figures in the same way. The driver script `demo_main.m` presents an interactive menu and runs demos individually or in sequence:

```
cd octave/examples
demo_main
```

The demos are:

- `demo_const_known`: smoothing with known parameters; posterior bands and empirical coverage check on a 2D state.

- `demo_tv_known`: time-varying parameters via the cell-array interface; a sinusoidally varying F_t illustrates the API.
- `demo_compare_recursive`: simultaneous smoother vs textbook Kalman + RTS, agreement to $\sim 10^{-15}$.
- `demo_missing`: NaN observations dropped automatically; posterior variance grows inside the gap.
- `demo_hp_filter`: the HP filter as a special case, matching the closed-form $(I + \lambda D_2^\top D_2)^{-1}y$ to within boundary effects.
- `demo_const_em`: EM with smoothed sufficient statistics; verifies that the marginal log-likelihood is monotone non-decreasing.
- `demo_robust`: 10% of observations contaminated with extra noise; Student- t and Huber cut RMSE by roughly 40 percent.
- `demo_robust_breaks`: structural-break detection via state-side robust prior; three injected level shifts identified at $\xi_t < 0.02$.
- `demo_hp_credit`: flagship demo on an HP filter with a credit-cycle exogenous driver, breaks, and outliers; compares Gaussian smoothing, Student- t robust smoothing at true parameters, and Gaussian EM for unknown (F, H, Q, R, B, D) as separate estimators (see Section 6).
- `demo_slds`: two-regime AR(1); exact variational EM recovers regime probabilities at $\sim 90\%$ classification accuracy. This demo uses a portable random-number generator so Octave, MATLAB, and Python simulate identical data and report identical results.
- `demo_constrained`: NAIRU-style non-negativity constraint via interior-point QP; constraint exactly satisfied while the unconstrained smoother goes negative in 31 of 120 periods.

The three worked applications of Appendix C are also implemented as runnable scripts, with filenames `appk_*.m`, in the `applications/` subdirectory of `octave/examples/`.

B Python implementation

The `python/` directory contains a direct port of the Octave implementation to Python, packaged as `simkssmoother`. The mathematics is identical to the Octave version; only surface conventions differ.

B.1 Installation

The recommended setup is the standard editable install:

```
cd python
pip install -e .
```

Requires Python 3.9 or later, NumPy 1.20+, and SciPy 1.7+, plus `matplotlib` for the demo figures. Optional: `scikit-sparse` (CHOLMOD bindings) for faster sparse factorization on large problems.

The demos and worked applications also run without `pip install -e .` by adding the `python/` directory to `sys.path` themselves when they detect that `simkssmoother` is not yet importable. This lets users explore the package straight from a freshly unzipped copy of the repository (e.g. via IPython's `run demo_const_known` from inside `python/examples/`).

B.2 Calling conventions

Functions are imported from the top-level package:

```
import simk smoother as sks
```

Required arguments come first; options are passed as keyword arguments rather than an `opts` struct. `Z`, `B`, `D` keyword arguments carry exogenous inputs. Diffuse priors are signalled by `P0=None` (instead of Octave's `P0=[]`). Missing observations are `np.nan` entries in `Y`. Returned arrays use the NumPy convention: rows are state coordinates, columns are time. Note the indexing offset relative to Octave: column `t` of `Xhat` corresponds to x_t ($t = 0, 1, \dots, T$), not column `t+1`.

B.3 Core smoothing and inference

`smooth_const.` Basic smoother.

```
Xhat, J, h, info = sks.smooth_const(  
    Y, F, H, Q, R, a0, P0,  
    Z=None, B=None, D=None, return_cov=False)
```

Inputs: the seven required state-space arguments; optional exogenous inputs `Z`, `B`, `D`; flag `return_cov`.
Outputs: the smoothed path `Xhat` ($n \times (T + 1)$ NumPy array), the SciPy sparse precision matrix `J` (CSC format), the information vector `h`, and `info` (a Python dict). With `return_cov=True`, `info` contains `Ptt` and `Ptt1`, lists of NumPy arrays for the diagonal and first-subdiagonal blocks of J^{-1} .

`smooth_tv.` Time-varying smoother.

```
Xhat, J, h, info = sks.smooth_tv(  
    Y, Fseq, Hseq, Qseq, Rseq, a0, P0,  
    Z=None, Bseq=None, Dseq=None, return_cov=False)
```

Each of `Fseq`, `Hseq`, `Qseq`, `Rseq` may be either a single NumPy array (constant) or a length- T list of arrays (per-period).

`selected_inv.` Selected inversion.

```
Ptt, Ptt1 = sks.selected_inv(J, n)
```

Inputs: the sparse precision matrix `J` and the block size n . *Outputs:* Python lists of (n, n) NumPy arrays for the diagonal blocks (length $T + 1$) and the first-subdiagonal blocks (length T) of J^{-1} .

`marginal_loglik.` Marginal log-likelihood.

```
ll, parts = sks.marginal_loglik(  
    Y, F, H, Q, R, a0, P0, Z=None, B=None, D=None)
```

Outputs: the scalar log-likelihood `ll` and a dict `parts` with the diagnostic breakdown.

`simulate_const.` Forward simulation.

```
Y, X = sks.simulate_const(  
    T, F, H, Q, R, a0, P0,  
    Z=None, B=None, D=None, rng=None)
```

Uses the modern `numpy.random.Generator` API; pass an `rng` for reproducibility or accept the default.

`rts_smoother`. Reference Kalman + RTS.

```
Xhat, P = sks.rts_smoother(Y, F, H, Q, R, a0, P0)
```

B.4 Parameter estimation

`em_const`. Gaussian EM.

```
params, Xhat, hist = sks.em_const(  
    Y, n, max_iter=200, tol=1e-6,  
    F=None, H=None, Q=None, R=None, a0=None, P0=None,  
    Z=None, B=None, D=None, verbose=False)
```

Inputs: the data and the state dimension n ; optional initializations and convergence controls; optional exogenous inputs. *Outputs:* `params` (dict with keys `F`, `H`, `Q`, `R`, `a0`, `P0`, and `B`, `D` if exogenous was supplied), the final smoothed path, and a list of marginal log-likelihood values across iterations (monotone non-decreasing).

B.5 Beyond-Gaussian and constrained smoothing

`smooth_robust`. Heavy-tailed IRLS smoother.

```
Xhat, info = sks.smooth_robust(  
    Y, F, H, Q, R, a0, P0, family,  
    nu=4.0, c=1.345, robust_state=False,  
    exact_estep=False,  
    max_iter=50, tol=1e-6,  
    Z=None, B=None, D=None)
```

`family` is one of "gaussian", "student-t", "laplace", "huber". By default the weights are plug-in IRLS (the fixed point is the posterior mode); `exact_estep=True` (Student- t only) switches to the covariance-corrected E-step of Section 7.1. `info` contains the final per-period weight vector `tau_v` (and `tau_w` when `robust_state=True`); weights much below 1 flag outliers or structural breaks.

`smooth_slds`. Switching dynamics.

```
Xhat, info = sks.smooth_slds(  
    Y, F_set, H, Q_set, R, a0, P0,  
    A=None, pi0=None, Z=None, B_set=None, D=None,  
    max_iter=30, tol=1e-5)
```

`F_set` and `Q_set` are length- K lists of NumPy arrays; regime-dependent state loadings go in `B_set` (length- K list of $n \times k$ arrays) alongside the common `Z` and the regime-invariant `D`. The routine implements the exact variational coordinate updates of Section 7.2. `info["pi"]` is the $K \times T$ regime-responsibility matrix at convergence; `info["Ptt"]` and `info["Ptt1"]` are the posterior covariance blocks of $q(X)$.

`smooth_constrained`. Inequality-constrained smoother.

```
Xhat, info = sks.smooth_constrained(  
    Y, F, H, Q, R, a0, P0,  
    A_ineq, b_ineq,  
    Z=None, B=None, D=None)
```

`A_ineq` should be a SciPy sparse matrix or a dense array of shape $(c, (T + 1)n)$; `b_ineq` is a length- c array. `info` contains the active-set mask, approximate KKT multipliers, and interior-point convergence diagnostics.

B.6 Demos and tests

The same eleven demos as the Octave package are provided in `python/examples/`, with filenames `demo*.py` matching the Octave counterparts one-for-one. Like their Octave counterparts, the Python demos print numerical diagnostics and produce `matplotlib` figures illustrating the same quantities. Figures are simultaneously saved to `python/examples/figures/` and displayed on screen when a GUI backend is available; on headless systems the demos fall back to the non-interactive Agg backend silently. The three worked applications of Appendix C produce figures in the same way. Run any demo individually, or use the interactive menu:

```
cd python/examples
python demo_main.py
```

Given identical inputs, the Python and Octave/MATLAB core routines agree to floating-point precision (for example, on the shared `demo_slds` data the smoothed paths agree to $\sim 10^{-16}$ and the regime responsibilities to $\sim 10^{-15}$). Note, however, that most demos draw their simulated data from each language’s *native* random-number generator, so their printed numbers differ across languages while telling the same qualitative story; `demo_slds` is the exception, using a portable generator so that all environments produce identical output. The headline numbers quoted in this note are from the Python runs. The three worked applications of Appendix C are likewise implemented as `appk*.py` in the `applications/` subdirectory of `python/examples/`.

A small test suite verifies the Python implementation:

```
cd python
python tests/test_all_demos.py
```

Five tests are run: simultaneous-vs-RTS agreement to $\sim 10^{-13}$; selected inversion vs dense inverse; exogenous-consistency ($B = D = 0$ reproduces the no-exogenous solution); EM marginal log-likelihood is monotone non-decreasing across iterations; and all eleven demos run without error.

C Three worked applications

The appendix walks through three end-to-end applications, each of which is a recognizable workhorse in applied macroeconomic work. The purpose is not to introduce new theory — everything follows from the machinery developed in Sections 4 through 7.3 — but to show, by repeated example, that the machinery handles substantively different problems with the same tools.

Each application is implemented as a runnable script in both Octave and Python, in the directories `octave/examples/applications/` and `python/examples/applications/`, with filenames `appk*.m` and `appk*.py` respectively. Running them reproduces the numbers reported here.

C.1 Natural rate of interest

The first application is the natural rate of interest r_t^* , the real short rate consistent with output at its trend. Following Laubach and Williams (2003) and Holston, Laubach, and Williams (2017) (“HLW”), we estimate r_t^* jointly with trend output y_t^* and trend growth g_t , using a small structural state-space model.

Model. Three latent objects evolve as random walks: trend output y_t^* , trend growth g_t , and an AR(2) output gap $\tilde{y}_t$. The natural rate is tied to trend growth through

$$r_t^* = c g_t, \quad (121)$$

the “Laubach–Williams identification” — a cross-equation restriction that pins down what would otherwise be an unidentified latent path. The state and measurement equations are

$$y_t^* = y_{t-1}^* + g_{t-1} + \varepsilon_t^{y^*}, \quad \varepsilon_t^{y^*} \sim N(0, \sigma_{y^*}^2), \quad (122)$$

$$g_t = g_{t-1} + \varepsilon_t^g, \quad \varepsilon_t^g \sim N(0, \sigma_g^2), \quad (123)$$

$$\tilde{y}_t = a_1 \tilde{y}_{t-1} + a_2 \tilde{y}_{t-2} + \frac{a_r}{2} \sum_{j=1}^2 (r_{t-j} - r_{t-j}^*) + \varepsilon_t^{\tilde{y}}, \quad (124)$$

$$y_t = y_t^* + \tilde{y}_t, \quad (125)$$

$$\pi_t - b_\pi \pi_{t-1} - (1 - b_\pi) \bar{\pi} = b_y \tilde{y}_{t-1} + v_t. \quad (126)$$

The IS curve uses the standard HLW timing: the real-rate *gap* enters as the average of its first two lags, so with $r_t^* = c g_t$ the equation expands to

$$\tilde{y}_t = a_1 \tilde{y}_{t-1} + a_2 \tilde{y}_{t-2} + \frac{a_r}{2} (r_{t-1} + r_{t-2}) - \frac{a_r c}{2} (g_{t-1} + g_{t-2}) + \varepsilon_t^{\tilde{y}} : \quad (127)$$

the observed policy-rate average enters as an exogenous input, and the natural-rate average appears as a state-side loading of $-a_r c/2$ on *each* of g_{t-1} and g_{t-2} — a single normalization used consistently in the equations, the transition matrix below, and the package code. The inflation equation is implemented in quasi-differenced form: the left-hand side $\pi_t - b_\pi \pi_{t-1} - (1 - b_\pi) \bar{\pi}$ is the observable fed to the smoother (so lagged inflation never needs to live in the state), and its loading b_y on $\tilde{y}_{t-1}$ is what identifies the Phillips slope.

State-space mapping. Augmenting the state to carry the necessary lags gives

$$x_t = (y_t^*, g_t, g_{t-1}, \tilde{y}_t, \tilde{y}_{t-1})^\top \in \mathbb{R}^5. \quad (128)$$

In $x_{t-1} = (y_{t-1}^*, g_{t-1}, g_{t-2}, \tilde{y}_{t-1}, \tilde{y}_{t-2})^\top$, trend growth g_{t-1} is the second component and g_{t-2} the third, so both lags the IS curve needs are already in the state. Reading the equations off row by row, the transition matrix is

$$F = \begin{pmatrix} 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & -\frac{a_r c}{2} & -\frac{a_r c}{2} & a_1 & a_2 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}, \quad B = \begin{pmatrix} 0 \\ 0 \\ 0 \\ a_r \\ 0 \end{pmatrix}, \quad (129)$$

with exogenous regressor $z_t = \frac{1}{2}(r_{t-1} + r_{t-2})$: the first row implements $y_t^* = y_{t-1}^* + g_{t-1}$ (the loading sits on the second component of x_{t-1} , i.e. g_{t-1} , matching the equation’s timing), and the fourth row carries the two $-a_r c/2$ loadings on g_{t-1} and g_{t-2} . Q is diagonal with *three* substantive innovation variances — $\sigma_{y^*}^2$, σ_g^2 , and $\sigma_{\tilde{y}}^2$ — plus tiny regularizers on the two deterministic lag-shift components (rows three and five carry no shock of their own); the inflation disturbance v_t is a *measurement* shock and lives in R , not Q . The observation matrix has output loading on y^* and $\tilde{y}$ and a quasi-differenced-inflation row that picks up $b_y \tilde{y}_{t-1}$.

Identification. Three issues are worth flagging:

- The IS-curve slope a_r is famously near zero on postwar US data (Holston, Laubach, and Williams, 2017), which causes a pile-up problem on σ_g when both are estimated. The simultaneous form does not resolve this — it is a fundamental identification limitation — but it makes the source visible: the EM-estimated likelihood becomes nearly flat in a direction of parameter space.
- Diffuse initialization on y^* and g is required (the random-walk trend has no proper stationary distribution). This is the $P_0 = \infty$ case treated in Section 4.8: the prior block of J is dropped.
- As an illustration of constrained smoothing, we impose $r_t^* \geq 0$ on the smoothed path: $-c g_t \leq 0$ for all t , plugged into the QP framework of Section 7.3 with one row of A_{ineq} per period. One clarification on the economics: this is a *maintained restriction on the natural rate*, not the zero lower bound — the ZLB constrains the *nominal policy rate* i_t , while the natural real rate r_t^* can perfectly well be negative in the model (and in the simulation below the true r^* sometimes is). A researcher might still maintain $r^* \geq 0$ as a structural prior for a particular economy; the point of the exercise is to show how cheaply such a restriction is imposed, and (below) what happens when it is wrong.

Results on simulated data. On 200 quarters of data generated from the model with $\sigma_g = 0.08$ (large enough that the true r^* visits negative values), the demo reports:

- Estimator A (Gaussian smoother, true parameters): RMSE on g_t is 0.23; the posterior standard deviation of r^* at the end of sample is 0.25, roughly 1.5 times its full-sample average of 0.17. This is the well-known result that r^* is more precisely identified *inside* the sample than at the latest date, where only one-sided information is available.
- Estimator B (constrained smoother): the unconstrained estimator places r^* below zero in 37 of 200 quarters; the constrained estimator pins $r^* \geq 0$ to machine precision. In this particular Monte Carlo draw the true r^* is itself negative in 27 quarters, so the maintained restriction contradicts the data-generating process and (mildly) *hurts* RMSE. This is the right diagnostic: an inequality constraint is informative only when it reflects truth. On samples where the true r^* stays non-negative (a more plausible empirical setting in some advanced economies), the constraint improves accuracy.
- Estimator C (EM, all parameters unknown): the EM-estimated y^* correlates with the true y^* at ~ 0.99 , with monotone marginal log-likelihood across iterations. Identification of the latent dynamics rotation is not perfect — F in the unrestricted EM is not unique, only its action on the data — but the decomposition into trend and cycle is well-recovered.

The point of this application is that the same three pieces of machinery developed earlier in the note — the basic smoother (Section 4), constrained smoothing (Section 7.3), and EM with exogenous inputs (Section 6) — combine without modification to deliver a full HLW-style analysis. No bespoke “natural-rate code” is needed.

C.2 Nowcasting current-quarter GDP

The second application is a mixed-frequency dynamic factor model for nowcasting current-quarter GDP from monthly indicators. This is the workhorse setup at central banks and IMF country desks; the canonical reference is Mariano and Murasawa (2003), with extensions to large panels by Giannone, Reichlin, and Small (2008) and Banbura and Modugno (2014).

Model. A single latent monthly factor f_t drives all indicators:

$$f_t = \phi f_{t-1} + e_t, \quad e_t \sim N(0, \sigma_e^2), \quad (130)$$

$$y_{k,t} = \lambda_k f_t + u_{k,t}, \quad u_{k,t} \sim N(0, R_{kk}), \quad k = 1, \dots, K. \quad (131)$$

Quarterly GDP enters as a third-month aggregate: at the end of each quarter,

$$\text{GDP}_t = \lambda_g \cdot \frac{1}{3}(f_t + f_{t-1} + f_{t-2}) + u_t^{\text{GDP}}, \quad (132)$$

and is unobserved (NaN) at all other monthly time stamps.

State augmentation. The third-month average requires the state to carry two lags of the factor: $x_t = (f_t, f_{t-1}, f_{t-2})^\top$, so $n = 3$. The observation matrix has K rows for the monthly indicators (each loading on the first component only) and one row for GDP:

$$H = \begin{pmatrix} \lambda_1 & 0 & 0 \\ \vdots & \vdots & \vdots \\ \lambda_K & 0 & 0 \\ \lambda_g/3 & \lambda_g/3 & \lambda_g/3 \end{pmatrix}. \quad (133)$$

Jagged edge. At any real-time vintage, the data panel has a “jagged edge”: different indicators have different publication lags (industrial production typically lags by 1 month, employment by 1 month, retail sales by 1-2 months, GDP by 1-2 quarters), so the last several observations are missing for some series and present for others. In the simultaneous formulation this is just a pattern of NaN entries; the smoother handles them by dropping rows of $H'R^{-1}H$ at each affected t . No bespoke “missing-data” code is needed.

Real-time nowcast experiment. At the end of each of the three months of a quarter, we re-run the smoother on the data available at that vintage and read off the current-quarter nowcast as the smoothed average of the factor over the three months. The posterior variance of this average is

$$\text{Var}(\widehat{\text{GDP}}) = \lambda_g^2 \cdot \frac{1}{9} \mathbf{1}^\top \Sigma_3 \mathbf{1}, \quad (134)$$

where Σ_3 is the 3×3 posterior covariance of the three factor values over the quarter — and here the state augmentation pays a small dividend. Because the augmented state at the end-of-quarter month t_3 is $x_{t_3} = (f_{t_3}, f_{t_3-1}, f_{t_3-2})^\top$, Σ_3 is the *single* diagonal block $\Sigma_{t_3 t_3}$ delivered by the Takahashi recursion of Section 4.5; its off-diagonal *elements* already contain all three cross-month covariances, including the lag-two term $\text{Cov}(f_{t_3}, f_{t_3-2} \mid Y)$. No off-diagonal time blocks of J^{-1} are needed for this formula (assembling Σ_3 from the scalar factor at three separate times would require them, and dropping the lag-two cross term — which is positive for a persistent factor — would understate the nowcast variance).

Results on simulated data. On 10 years of monthly data with $K = 5$ indicators (release lags $[0, 1, 1, 0, 0]$) and one quarterly GDP series, the nowcast standard deviation shrinks from 0.36 with one month of data to 0.21 with two months to 0.12 at the end of the quarter. The factor itself is recovered with 0.96 correlation against truth despite the missing-data pattern. Nowcast point estimates fall within one standard deviation of the true value at every vintage.

The point of this application is the modularity claim from Section 4.7: jagged edges, mixed frequencies, and publication lags are all the same thing — patterns of missing observations — and the simultaneous smoother handles them with no special-case code. The posterior covariance from selected inversion gives the calibrated nowcast standard error as a free byproduct.

C.3 Semi-structural three-equation New Keynesian model

The third application is the workhorse three-equation New Keynesian model used for semi-structural projections in policy institutions (see [Berg, Karam, and Laxton \(2006\)](#) for the IMF reference implementation). Three equations link the output gap, inflation, and the policy rate:

$$\tilde{y}_t = \alpha_1 \tilde{y}_{t-1} + \alpha_2 (i_{t-1} - \pi_{t-1} - \bar{r}) + \varepsilon_t^y, \quad (135)$$

$$\pi_t = \beta_1 \pi_{t-1} + (1 - \beta_1) \bar{\pi} + \beta_2 \tilde{y}_{t-1} + \varepsilon_t^\pi, \quad (136)$$

$$i_t = \rho_i i_{t-1} + (1 - \rho_i) [\bar{r} + \bar{\pi} + \phi_\pi (\pi_{t-1} - \bar{\pi}) + \phi_y \tilde{y}_{t-1}] + \varepsilon_t^i. \quad (137)$$

The structural slopes $(\beta_2, \alpha_2, \phi_\pi, \phi_y)$ are the empirically interesting parameters; the structural shocks $(\varepsilon^y, \varepsilon^\pi, \varepsilon^i)$ are demand, cost-push, and monetary shocks, the bread and butter of policy narratives.

State-space mapping. The state $x_t = (\tilde{y}_t, \pi_t, i_t)$ contains everything an observer needs. All forward expectations have been replaced by their backward-looking reduced form; the system is purely autoregressive with three innovations. The F matrix is

$$F = \begin{pmatrix} \alpha_1 & -\alpha_2 & \alpha_2 \\ \beta_2 & \beta_1 & 0 \\ (1 - \rho_i)\phi_y & (1 - \rho_i)\phi_\pi & \rho_i \end{pmatrix}, \quad B = \begin{pmatrix} -\alpha_2 \bar{r} \\ (1 - \beta_1) \bar{\pi} \\ (1 - \rho_i)(\bar{r} + \bar{\pi} - \phi_\pi \bar{\pi}) \end{pmatrix}, \quad (138)$$

with the exogenous regressor $z_t = 1$ carrying the constants. Observation is direct: $y_t = x_t + v_t$, with measurement error.

Shock decomposition. Given smoothed states $\hat{X}$, the smoothed structural shocks fall out:

$$\hat{\varepsilon}_t = \hat{x}_t - F \hat{x}_{t-1} - B z_t. \quad (139)$$

These are the central-bank-narrative quantities: "how much of last quarter's output gap was demand vs monetary?" The simultaneous smoother delivers them in one sparse solve.

Results on simulated data. On 200 quarters of simulated data with conventional calibration ($\beta_2 = 0.15$, $\rho_i = 0.80$, $\phi_\pi = 1.50$, $\phi_y = 0.50$):

- The Gaussian smoother with true parameters recovers all three structural shocks with correlations of 0.90, 0.99, and 0.97 against truth, respectively (demand, cost-push, monetary).
- EM with all parameters unknown converges in ~ 120 iterations with monotone marginal log-likelihood. The Phillips slope β_2 (i.e. κ , the canonical hard parameter) is recovered at 0.17 vs. a truth of 0.15. The Taylor-rule coefficients are noticeably less precise (ϕ_π at 1.87 vs. truth 1.50), a well-known consequence of the limited variation in policy-rate movements available in the simulated sample: with the smoothing parameter $\rho_i = 0.80$, the systematic part of the rule contributes little independent variation, and the likelihood is correspondingly flat in (ϕ_π, ϕ_y) .

No DSGE-specific estimation routine is involved. The state-space formulation plus the standard EM algorithm of Section 6 delivers structural identification, shock decomposition, and posterior bands on the latent gaps on the same sparse J that the basic smoother used.

C.4 What the three applications have in common

The point of the three together is conceptual unity. Each application defines (F, H, Q, R, B, D) for a substantively different economic problem. Each produces a substantively different output: r^* at the latest date, current-quarter GDP, structural shock decomposition. Each exercises a different optional capability of the framework: constrained smoothing, missing data, EM on a multi-equation system. But the core machinery is identical. The work-horse object in every case is the same sparse block-tridiagonal linear system $JX = h$, solved once or iterated as part of EM, with the same selected-inversion machinery for posterior covariances. This is what is meant by “the right idiom” for state-space modelling in macroeconometrics.

References

- Andrews, D. F., and C. L. Mallows (1974). “Scale mixtures of normal distributions.” *Journal of the Royal Statistical Society, Series B*, 36(1):99–102.
- Banbura, M., and M. Modugno (2014). “Maximum likelihood estimation of factor models on datasets with arbitrary pattern of missing data.” *Journal of Applied Econometrics*, 29(1):133–160.
- Berg, A., P. Karam, and D. Laxton (2006). “Practical model-based monetary policy analysis: a how-to guide.” *IMF Working Paper* WP/06/80.
- Bierman, G. J. (1977). *Factorization Methods for Discrete Sequential Estimation*. Academic Press, New York.
- Botev, Z. I. (2017). “The normal law under linear restrictions: simulation and estimation via minimax tilting.” *Journal of the Royal Statistical Society, Series B*, 79(1):125–148.
- Carpenter, B., A. Gelman, M. D. Hoffman, D. Lee, B. Goodrich, M. Betancourt, M. Brubaker, J. Guo, P. Li, and A. Riddell (2017). “Stan: a probabilistic programming language.” *Journal of Statistical Software*, 76(1):1–32.
- Chan, J. C. C., and I. Jeliazkov (2009). “Efficient simulation and integrated likelihood estimation in state space models.” *International Journal of Mathematical Modelling and Numerical Optimisation*, 1:101–120.
- Davis, T. A. (2006). *Direct Methods for Sparse Linear Systems*. SIAM.
- de Jong, P. (1991). “The diffuse Kalman filter.” *Annals of Statistics*, 19(2):1073–1083.
- Durbin, J., and S. J. Koopman (2012). *Time Series Analysis by State Space Methods*, 2nd ed. Oxford University Press.
- Erismann, A. M., and W. F. Tinney (1975). “On computing certain elements of the inverse of a sparse matrix.” *Communications of the ACM*, 18(3):177–179.
- Ghahramani, Z., and G. E. Hinton (2000). “Variational learning for switching state-space models.” *Neural Computation*, 12(4):831–864.
- Giannone, D., L. Reichlin, and D. Small (2008). “Nowcasting: the real-time informational content of macroeconomic data.” *Journal of Monetary Economics*, 55(4):665–676.

- Giannone, D., M. Lenza, and G. E. Primiceri (2015). “Prior selection for vector autoregressions.” *Review of Economics and Statistics*, 97(2):436–451.
- Holland, P. W., and R. E. Welsch (1977). “Robust regression using iteratively reweighted least-squares.” *Communications in Statistics: Theory and Methods*, 6(9):813–827.
- Holston, K., T. Laubach, and J. C. Williams (2017). “Measuring the natural rate of interest: international trends and determinants.” *Journal of International Economics*, 108(S1):S59–S75.
- Kalman, R. E. (1960). “A new approach to linear filtering and prediction problems.” *Journal of Basic Engineering*, 82(D):35–45.
- Koopman, S. J., and J. Durbin (2003). “Filtering and smoothing of state vector for diffuse state space models.” *Journal of Time Series Analysis*, 24:85–98.
- Lange, K. L., R. J. A. Little, and J. M. G. Taylor (1989). “Robust statistical modeling using the t -distribution.” *Journal of the American Statistical Association*, 84(408):881–896.
- Laubach, T., and J. C. Williams (2003). “Measuring the natural rate of interest.” *Review of Economics and Statistics*, 85(4):1063–1070.
- Mariano, R. S., and Y. Murasawa (2003). “A new coincident index of business cycles based on monthly and quarterly series.” *Journal of Applied Econometrics*, 18(4):427–443.
- McCausland, W. J., S. Miller, and D. Pelletier (2011). “Simulation smoothing for state-space models: A computational efficiency analysis.” *Computational Statistics & Data Analysis*, 55(1):199–212.
- Rauch, H. E., F. Tung, and C. T. Striebel (1965). “Maximum likelihood estimates of linear dynamic systems.” *AIAA Journal*, 3(8):1445–1450.
- Roweis, S., and Z. Ghahramani (1999). “A unifying review of linear Gaussian models.” *Neural Computation*, 11(2):305–345.
- Rue, H., and L. Held (2005). *Gaussian Markov Random Fields: Theory and Applications*. Chapman & Hall/CRC.
- Rue, H., S. Martino, and N. Chopin (2009). “Approximate Bayesian inference for latent Gaussian models by using integrated nested Laplace approximations.” *Journal of the Royal Statistical Society, Series B*, 71(2):319–392.
- Shumway, R. H., and D. S. Stoffer (1982). “An approach to time series smoothing and forecasting using the EM algorithm.” *Journal of Time Series Analysis*, 3(4):253–264.
- Takahashi, K., J. Fagan, and M. Chen (1973). “Formation of a sparse bus impedance matrix and its application to short circuit study.” *Proc. 8th PICA Conference*, pp. 63–69. Minneapolis.
- Watson, M. W., and R. F. Engle (1983). “Alternative algorithms for the estimation of dynamic factor, MIMIC and varying coefficient regression models.” *Journal of Econometrics*, 23(3):385–400.